
Audit the Scaffold, Not the Checkpoint: A Stationarity Dichotomy for Recursive Self-Improvement in Agentic Coding

Sebastian Bobadilla-Suarez*
OnCorps
sebastian.suarez@oncorps.io

Bob Suh
OnCorps
bob@oncorps.io

Ryan Fortin
OnCorps
ryan@oncorps.io

Abstract

Rules for AI self-improvement often key on the model itself: has it been retrained, or are its weights (the trained model) frozen? We argue that, for AI coding agents, this check is incomplete, and we propose, but do not run, the experiment that would test the argument. The criterion is simple. An agent that keeps revising its own work runs into diminishing returns as long as the set of changes it can reach stays fixed, and can escape them only if that set grows. Our hypothesis is that the set can grow with the model untouched, because the software around the model, its *scaffold* of tools, tests and task breakdown, can be rewritten. If so, a frozen model guarantees a ceiling eventually but not steady behavior on the way there, and audits should track changes to the scaffold, not only to the model. The same criterion separates three things often lumped together: more search within a fixed setup, retraining the model during use (which can raise the ceiling), and rewriting the scaffold, which sits between them.

Adding more agents does not lift the ceiling either. Picking the best of several agents' answers reaches the best single agent's ceiling exactly, and only gets there faster. A majority vote could do better, but only if the agents fail on different tasks. In our tests, 30 agents from one model family failed together: some task defeated all of them, and a majority failed 23/55 (42%) of tasks.

Our evidence is descriptive. On a public coding benchmark, the gain from each extra round of revision shrinks toward zero, though the unsolved tasks left to measure are also fewer and harder. Across 401 real development sessions, the amount of code changed falls off steadily from commit to commit, but programmers working before AI tools show the same pattern, so the data show that current practice levels off without showing why. We reach the criterion by treating repeated revision as a known machine-learning method, boosting, and report where real agents depart from it. Those departures are design choices, and together they describe an agent setup worth building.

1 Introduction

AI systems now write much of their own code. Anthropic reports that more than 80% of the code merged into its production codebase was authored by Claude as of May 2026, up from low single digits before its coding-agent preview launched in February 2025, with engineers increasingly supplying the goal rather than the method [Favaro and Clark, 2026]. In *agentic coding*, one or more AI agents draft code, test it, and revise it in a loop. That loop is a small, observable version of the scenario behind worries about recursive self-improvement (RSI), in which a system improves itself

*Corresponding author.

faster and faster [Good, 1965, Bostrom, 2014]. This paper asks two questions: *when does a loop of self-revision run out of room, and is a frozen model evidence that it has?*

The argument in brief. Picture a craftsman whose hands are fixed but whose workshop is not. A sharper tool lets the same hands reach further, yet no rearrangement of the workshop makes the hands more dexterous. An AI coding agent is like this: the trained model (its *weights*, or *checkpoint*) is the hands, and the *scaffold*, the tools, tests, prompts and task breakdown the model works inside, is the workshop. Our criterion (§4) says that self-revision levels off whenever the set of changes the agent can reach stays fixed, and can keep accelerating only if that set keeps growing. Our hypothesis is that rewriting the scaffold can grow the set while the model stays frozen. If it can, a frozen model guarantees only an eventual ceiling, not steady behavior on the way to it, and an auditor who checks only whether the model changed is checking too little. Current safety frameworks already test models with good scaffolding, but they schedule re-assessment around training events, so a scaffold rewritten between assessments goes unexamined (§6).

What this paper does and does not show. We reach the criterion by viewing repeated revision through *boosting*, a classic machine-learning method in which weak predictors are combined, each correcting the errors left by the last. Each agent edits an existing draft rather than writing from scratch, much as each boosting step corrects the remaining error. The lens is a tool, not the contribution: the criterion itself does not depend on boosting and is elementary to prove. Its value is in naming what to track. Comparing real agent loops against the lens shows two places where they depart from it, and both are design choices. The empirical part is *descriptive*. Current agentic coding does level off, on a benchmark and in production, but human programmers working before AI tools level off in the same way, so the data cannot show that a fixed reachable set is the *cause* (§5.2). We also cannot measure the reachable set directly. §6 proposes a practical stand-in and the experiment that would test the hypothesis: hold the model fixed and vary only the scaffold. We do not run it. What follows is a criterion and a hypothesis, not a demonstrated mechanism.

Contributions.

- **A criterion for auditors.** Repeated revision with a bounded quality score must slow to zero gains (Thm. 2); it slows whenever the reachable set of edits is fixed and can escape only if that set grows (Prop. 3). If scaffold rewrites can grow it with the model frozen, a frozen model guarantees an eventual ceiling but not steady behavior along the way (Cor. 4). Growth is necessary but not sufficient for escape, so the check errs toward false alarms. Current practice levels off, on SWE-bench and across 401 production sessions, but so does a 9,395-session pre-AI baseline, so the cause is left open (§5.1, §5.2).
- **More agents do not raise the ceiling.** Picking the best of k agents reaches the best agent’s ceiling exactly and no higher (Prop. 5). A majority vote could do better only if agents fail on different tasks (Prop. 6); ours, from one model family, fail together, and a majority fails 23/55 (42%) of tasks. Asking the same pool again each round also has a limit that can be computed in advance (Prop. 7).
- **Where real agent loops depart from the lens.** They overwrite each draft instead of keeping and weighing every attempt, and their failures coincide. Both are design choices, so together they describe an agent setup worth building, which nobody, ourselves included, has built yet (§3).
- **A proposed decisive experiment.** Hold the model fixed, add scaffold capabilities one at a time, and measure both the per-round gains and how many tasks the agent can solve at all within many tries (§6). We specify it; we do not run it.

2 Related Work

Self-improvement and scaffolds. The “intelligence explosion” [Good, 1965, Bostrom, 2014] and the Gödel machine [Schmidhuber, 2003] motivate the RSI reading. Systems that improve code with the model held fixed are the case our ceiling addresses: STOP [Zelikman et al., 2023], FunSearch [Romera-Paredes et al., 2024], and the Darwin Gödel Machine [Zhang et al., 2025a], which rewrites its own scaffold with frozen weights. The scaffold, or *harness*, is increasingly treated as a layer to optimize in its own right [He et al., 2026]. Xue and Yang [2026] let a frontier model design its own improvement process as it runs, close to pure scaffold expansion, and find it beats staged

and reflective-evolutionary pipelines [Agrawal et al., 2026] with a strong model but *loses* with a medium one. Cor. 4 predicts that shape: scaffold expansion pays only as far as the frozen model allows. A recent survey splits RSI into convergent *bounded self-refinement* and *open-ended* RSI [Chen et al., 2026]; our criterion (Prop. 3) gives that split a structural cause. Empirical multi-agent and self-refinement systems, and the test-time-compute literature whose diminishing returns our model predicts, are surveyed in Appendix G.1.

The boosting lens. AdaBoost [Freund and Schapire, 1997] and its margin theory [Schapire et al., 1998, Schapire, 2013] supply our vocabulary. Reading boosting as gradient descent on the remaining error [Mason et al., 1999, Friedman, 2001] is what lets a code patch play the role of a correction step, and mapping a specification to code is an instance of structured prediction [Tsochantaridis et al., 2005].

Coverage is not a vote. Our overlap result could be mistaken for a known one. Repeated sampling measures *coverage*, whether *any* of k attempts solves a task, and coverage keeps growing with k [Brown et al., 2024]. That assumes something can recognize the good attempt afterwards. Without such a checker an orchestrator falls back on a vote, and Prop. 6 governs the vote’s error: the tasks a vote cannot recover are exactly those most workers fail. Growing coverage and a failing vote are therefore consistent. Cross-vendor aggregators [Wang et al., 2024, Jiang et al., 2023] report end-to-end gains but not the overlap statistic that decides whether a vote can work (open problem 2).

Three cases usually merged. Extra inference compute searches within what the model and scaffold already allow. Test-time *training* updates the model during use [Sun et al., 2020, Hardt and Sun, 2024, Sun et al., 2025, Akyürek et al., 2025, Hübotter et al., 2025], as does letting the model’s own generated edits drive an update [Zweiger et al., 2025], and so can raise the ceiling itself. Between them sits this paper’s case: rewriting the scaffold with the model frozen. The separation matters for oversight: an auditor who checks only whether the model changed catches test-time training and misses scaffold expansion entirely (Appendix G.4).

3 A Boosting Lens on Agentic Coding

In boosting, a sequence of *weak learners*, each only slightly better than chance, is combined into a strong one: each new learner concentrates on the examples the previous ones still get wrong, and the final answer is a weighted vote of all of them. The analogy to agentic coding is direct. A *weak coding agent* does not write the program from scratch; it proposes a patch (a git diff) that corrects the remaining error between the current draft and the target. Even a small edge over chance is demanding here, since a random edit almost always breaks a working program (Remark 14). Appendix A (Table 1) maps the two settings term by term.

Where real loops depart from boosting. Two departures matter. First, *real loops overwrite; boosting keeps*. A coding loop applies each patch on top of the last, so the newest draft replaces its predecessors ($R_T \circ \dots \circ R_1$). Boosting keeps every learner and lets them vote, and it tracks which examples are still failing so it can focus on them. Real loops do neither (two further gaps, and whether task decomposition might supply them implicitly, in Appendix G.2). Second, *real agents fail together*. A vote helps only if workers make different mistakes, and on our pool they do not: every round has some task that defeats every worker (§4).

Both are design choices, so they describe a better harness. A loop that kept every attempt, focused on the specifications still failing, and used workers diverse enough that fewer than half fail any task would satisfy boosting’s guarantees by construction (Algorithm 1; Prop. 6 makes the diversity condition exact). We do not build that system and know of no one who has (open problem 1).

What the lens buys. It supplies the vocabulary that makes “will this run away?” a structural question rather than a metaphor; it names the quantity to track, the set of reachable edits and its ceiling; and measuring where real loops depart from it produced the harness specification above and the orchestration results of §4.

4 Theoretical Analysis

This section states the criterion formally. The proofs are short, and that is deliberate: the claim is not that saturation was hard to prove but that the *reachable set of edits* is the right thing to track. The algebra is machine-checked in Lean 4 (Mathlib, no axiom, no sorry), and the check earned its keep by *refuting* one conjecture of ours (Prop. 5). It cannot check what matters most: the modeling assumptions (M1), (M2) and the boosting mapping itself (Remark 16).² Full proofs are in Appendix B.

Setup. A specification $x \in \mathcal{X}$ (a ticket, a failing test) is to be turned into code; drafts and the target y live in $\mathcal{Y}$, with a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$ and a loss $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ such as an edit distance (Table 1).

Definition 1 (Weak Coding Agent). A weak coding agent *proposes a patch via a refinement operator* $R_t : \mathcal{Y} \times \mathcal{A} \rightarrow \mathcal{Y}$, taking a draft z and an edit $a \in \mathcal{A}$ to $z' = R_t(z, a)$; let $h_t(x) \in \{-1, +1\}$ indicate success on specification x , $h_t(x) = +1$ iff the draft returned matches the target, $L(y, z') = 0$. Every label is $+1$ here, the target being solved by definition, so $\text{err}_{\mathcal{D}}(h_t) = \Pr[h_t(x) = -1] \leq \frac{1}{2} - \gamma_t$ and the appendix’s generic y_i reads as $+1$ throughout; $\mathcal{H}$ is instantiated only in Theorem 12. This γ_t is the edge on solving, the one Thm. 10 and Cor. 11 require; Part B measures the edge on improving the previous draft, written $\tilde{\gamma}_t = \frac{1}{N} \sum_i \mathbf{1}[q_{i,t} > q_{i,t-1}] - \frac{1}{2}$ on the pass fraction $q_{i,t}$ of task i at round t , which scores a tie at $-\frac{1}{2}$ under the strict $>$ (§5.1). The two need not agree, and which is reported shapes the headline.

The boosting reading, taken literally (Def. 9), would carry over three classical guarantees (Appendix B.1). We rely on none of them, since each needs a kept-and-reweighted ensemble that no real loop maintains. The results below make no use of boosting.

The criterion. Suppose every round of revision raises a quality score that cannot exceed 1. The gains then have to add up to at most the room left, so they must shrink toward zero:

Theorem 2 (Monotone Improvement under Refinement). Write $z_t = R_t(z_{t-1}, a_t^*)$ for the draft after round t at that round’s best action a_t^* , and suppose each R_t satisfies the improvement property $\mathbb{E}_{y \sim \mathcal{D}(\cdot|x)}[V(R_t(z, a_t^*))] \geq V(z) + \eta_t$ for quality function $V : \mathcal{Y} \rightarrow [0, 1]$ and $\eta_t > 0$. Then $\mathbb{E}[V(z_T)] \geq V(z_0) + \sum_{t=1}^T \eta_t$, and since $V \leq 1$, the total improvement saturates: $\sum_{t=1}^{\infty} \eta_t \leq 1 - V(z_0)$.

That already implies diminishing returns, but only if the ceiling stays put. The next result says what happens when it does not, and it is the paper’s central criterion:

Proposition 3 (Stationarity Dichotomy). Consider the refinement game of Thm. 2. (i) **Fixed ceiling** $\Rightarrow$ **saturation**. If $V(z) \leq B$ for all z , then $\sum_{t=1}^{\infty} \eta_t \leq B - \mathbb{E}[V(z_0)]$, forcing $\eta_t \rightarrow 0$: strict diminishing returns. (ii) **Sustained edge** $\Rightarrow$ **divergence**. If the effective hypothesis class expands enough to sustain a uniform edge $\eta_t \geq c > 0$, then $\mathbb{E}[V(z_T)]$ grows without bound and no fixed ceiling can hold.

Both parts are verified in Lean (Proposition8.lean) and follow almost immediately from Thm. 2. Part (i) is the bounded, saturating regime; part (ii) is what an unbounded “intelligence explosion” would require. The real difficulty sits in the assumptions, above all that quality has a fixed maximum, which a genuinely open-ended objective would deny (Appendix G.5). The safety question is then whether the reachable set can grow while the model stays the same:

Corollary 4 (Frozen-Weight Ceiling). Model a self-improving system as $\mathcal{S} = (W, C)$, frozen weights W and mutable scaffold C . Assume (M1) class expandability: the effective hypothesis class $\mathcal{H}_t(C)$ can expand under a scaffold rewrite without altering W ; and (M2) ultimate ceiling: $V(z) \leq V^*(W)$ for all z reachable by any scaffold the model can construct. Then (i) the system saturates at a value no greater than $V^*(W)$ (Prop. 3(i) with $B = V^*(W)$); (ii) escaping this ceiling requires raising $V^*(W)$ itself—modifying weights or architecture—not only rewriting the scaffold (contrapositive of Prop. 3(ii)).

²The formalization is a quality-control layer against AI-hallucinated derivations rather than a contribution: this manuscript was drafted with AI assistance (Responsible Use Statement).

So freezing the model does *not* by itself place a system in the safe, saturating regime: under (M1) the reachable set can grow while W is fixed, and the frozen model guarantees only the eventual ceiling of part (i). Neither (M1) nor (M2) is a theorem; both are explicit assumptions (Remark 16). The criterion also runs one way only. Escape needs a gain that stays above a fixed floor, so a scaffold that expands while its gains shrink still levels off: growth is necessary for escape but not sufficient. Appendix G.4 works one example, an agent that writes a test-isolation script and reuses it in later rounds, which unlocks patches it could not previously check, with the model untouched.

More agents: the ceiling binds sideways too. An orchestrator that runs k frozen workers, each under its own scaffold C_i (different prompts, tools or roles, possibly the same model), faces the same ceiling. Write $V^*(W, C_i) := \sup\{V(z) : z \text{ reachable under } C_i\}$ for worker i 's ceiling.

Proposition 5 (Orchestration Selects a Ceiling; It Does Not Raise One). *Best-of- k selection attains $V_{\text{orch}}^* = \max_i V^*(W, C_i)$ exactly—an unconditional equality, holding even when no single worker's reachable class contains the union of all of them. Consulting all k workers every round of the refinement game gives $\mathbb{E}[V(z_T)] \geq V(z_0) + \sum_{t=1}^T \max_i \eta_t^{(i)}$, dominating every worker's depth-only guarantee, yet since $V \leq 1$ the total improvement still saturates at $1 - V(z_0)$: width buys rate, not budget. (The width half needs one hypothesis beyond Thm. 2's—monotonicity of the inner action-expectation, since selection compares two integrands under the same expectation; Prop. 18 states it and the formalization flags it.)*

Picking the best answer therefore chooses which existing ceiling to reach, and reaches it sooner; it does not create a higher one. That makes “upgrade the model before adding orchestration” a consequence rather than a rule of thumb. We had conjectured that selection could strictly beat the best worker, and Lean found the counterexample we had missed: two workers that each reach one different answer of equal value.

A vote needs workers who fail on different tasks. Voting is the one arrangement that can beat the best single worker, because voters answer the same question side by side (Prop. 17). What it needs is diversity, and the requirement reduces to counting:

Proposition 6 (Bounded Overlap Characterizes a Correct Vote). *Let k workers with ± 1 outputs be combined by the unweighted majority vote, and let $m_i := \#\{t < k : y_i h_t(x_i) = -1\}$ count the workers that fail point i , with $m^* := \max_i m_i$ the tight overlap. Then the vote has zero training error iff $2m_i < k$ for every i —equivalently, iff $m^* < k/2$. The condition is sharp in two senses: a single point at $2m_i = k$ already makes the error nonzero, and if every point sits there the error is 1.*

The proposition is bookkeeping; the measurement carries the content, because it turns “is this pool diverse enough to vote?” into one countable number. Part B's pool fails at the extreme: in *every* round $m^* = k$, so some task defeats every worker, and a majority fails 23/55 (42%) of tasks, which is the vote's error (0.418, 95% CI [0.291, 0.545]). The vote's error also exceeds the mean single-worker error 0.356, but that gap's interval includes zero (+0.062, 95% CI [−0.018, +0.142]) and vanishes within one capability tier, so we do not read it as a fact about voting (Appendix C).

This is close to the weakest diversity test possible: one vendor, two tiers, three prompt variants at temperature 0.25, with the prompt contrast itself null (§5.1), so the pool's effective diversity is nearer two configurations than 6. It shows that resampling one model family inherits that family's blind spots. Whether workers from genuinely different vendors would clear the bar is open, and this data cannot reach it.

Asking the same pool again has a limit set in advance. An orchestrator usually consults its workers again each round. How long that can pay is decided by one property of the pool: whether some weighted combination of its workers can already get every task right.

Proposition 7 (Reused-Pool Dichotomy). *Fix a pool of k workers and run Algorithm 1 over it under any schedule at the optimal confidences, with every scheduled worker having edge $\gamma > 0$ against the distribution current when it is used. Let ϵ_0 be the smallest zero-one training error attained by any nonnegative combination of the pool. (i) If $\epsilon_0 > 0$, then $\epsilon_0 \leq e^{-2\gamma^2 T}$, hence $T \leq \log(1/\epsilon_0)/2\gamma^2$ —the edge hypothesis provably fails by a finite, computable round, uniformly over the schedule and the confidences, and depending on k only through ϵ_0 . (ii) If instead the pool interpolates ($\epsilon_0 = 0$) with margin θ at coefficient sum S , then for every distribution some member has edge $\gamma = \theta/2S > 0$,*

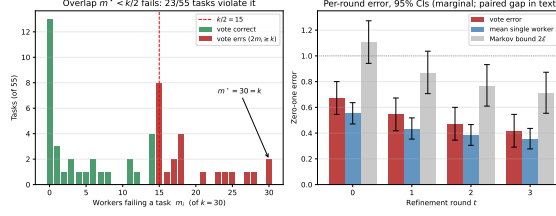


Figure 1: Prop. 6’s overlap condition measured on $k=30$ workers (6 arms $\times$ 5 seeds), shared 55-task pool. **Left:** workers failing each task; the condition needs every task strictly left of the dashed $k/2$ line, so the red mass is the vote’s error and the spike at $m_i=k$ is total overlap. **Right:** per round, vote error against mean per-worker error $\bar{\epsilon}$ and the Markov bound $2\bar{\epsilon}$.

fixed before the distribution is chosen, so the hypothesis survives every round of any run. The cases are exhaustive and mutually exclusive.

A fixed pool is therefore not a renewable resource: unless it can already fit the work, consulting it again has a limit set by ϵ_0 and γ , not by how much compute an orchestrator is willing to spend. And which side a pool falls on is, again, a property of the scaffold rather than of any weight. Both halves are Lean-checked (Props. 20 and 21).

5 Experiments

Three parts. **Part A** only checks that the boosting machinery is implemented correctly, on synthetic labels, and claims nothing about real agents (Appendix D). **Part B** runs a real coding agent on a public benchmark. **Part C** looks at 401 real development sessions. All code, processed data, figure inputs and the raw model response cache are in the supplementary repository.

5.1 Part B: An Agent Revising Its Own Work on SWE-bench Lite

Design. The agent is *mini-swe-agent* [Yang et al., 2024], given $T=4$ rounds to fix each of $K=55$ real GitHub issues from SWE-bench Lite [Jimenez et al., 2024], with 5 independent runs per task at temperature 0.25. Pilot runs showed the improvement rate collapsing by round 1, which set T . We cross two factors (1,650 runs in all). *Model tier:* Claude Sonnet (stronger) vs. Claude Haiku 4.5 (weaker). *Feedback:* what each later round is told about the previous attempt, either nothing (**independent**), the previous patch only (**blind**), or the previous patch plus which target tests still fail (**diagnostic**). No arm ever sees the correct patch.

Scoring, and a bug we found in it. Each attempt is scored by actually running the repository’s tests in a pinned Docker container: real pass/fail, not an AI judge. The official SWE-bench harness scores only once at the end, and we needed a score after every round, so we wrote our own scorer. Before trusting it we ran it on the *known-correct* patches, and 23/50 (46%) of the originally sampled $K=50$ tasks still failed, across 5 root causes in *our* scorer, each of which would otherwise have been counted against the model. We fixed the two shared causes, re-verified against the correct patches, and rebuilt the pool to $K=55$. The resulting pool favors repositories our scorer handles, so it is plausibly easier than SWE-bench Lite as a whole (Appendix E).

Model strength matters a lot; feedback type shows no detectable effect. Model tier explains roughly half the variance ($F(1, 54)=53.46$, $p<0.0001$, partial $\eta^2=0.497$; full ANOVA in Appendix E). Haiku solves 44–49% of tasks across arms and Sonnet 85–89% (a task counts as solved when $\geq \tau=0.6$ of its tests pass; 95% Wilson CIs). Neither feedback type ($p=0.61$) nor the interaction ($p=0.45$) is significant, but at this sample size the test has only 11%/17% power, and 80% power would need 7–9 $\times$ the tasks. *The feedback null is underpowered, not evidence of no effect.*

These are not SWE-bench resolve rates, since our τ counts the *fraction* of tests passing. Under SWE-bench’s own all-tests criterion ($\tau=1.0$) the same analysis gives Haiku 29–38% and Sonnet 71–75%: the tier gap survives, and the weaker tier’s feedback ordering flips, one more reason to read the feedback contrast as noise. The tier *contrast* is the result that should transfer, not the absolute rates.

Later rounds mostly change nothing. Most rounds after the first leave the code as it was: ties make up 80–95% of round-to-round transitions overall, and 13–21% (Sonnet) and 52–59% (Haiku) after excluding rounds that were never run (below), while at most 2.9% of rounds make things worse. The loop is wasteful rather than harmful: it acts less and less, not wrongly. On tasks that could still improve, the improvement *edge* (the share of rounds that improve, minus one half, so that zero is boosting’s weak-learning threshold) is Haiku (-0.345, -0.435, -0.411) on 194→158 of 275 rows and Sonnet (-0.084, -0.143, -0.276) on 113→49 of 275. Sonnet’s first two intervals include zero, so we do not claim the strong model falls below the threshold (Appendix E.1, Fig. 6).

One measurement is distorted by the harness, and we set it aside. The loop stops calling the model once a task passes and fills the remaining rounds with a perfect score. A simple “did this round improve?” rate then counts every filled-in round as a failure to improve, so pooled over all rounds it tracks how many tasks were solved, and backwards: the stronger model solves more, fills in more, and looks worse. We report that pooled series only for disclosure (Appendix E.1). Nothing in §4 depends on it.

The headline number, and its two confounds. The quantity that bears on the criterion is the mean gain per round on each task’s best score so far. It falls $0.097 \rightarrow 0.036 \rightarrow 0.033$ (Haiku) and $0.156 \rightarrow 0.068 \rightarrow 0.029$ (Sonnet): gains continue but shrink toward zero, the shape Thm. 2 predicts. Filled-in rounds do not bias this number the way they bias the improvement rate, since they add exactly zero gain. But two confounds sit beside it, and either could produce part of the decline without any ceiling. First, the mean runs over all tasks, so solved tasks add zeros and the set of tasks still able to improve shrinks and gets harder with each round. Second, because the loop stops on success, no round after a first pass was ever run, so this is not an unconditional per-round curve. A fixed set of tasks with every round executed would remove both (§6.1). This is also one vendor’s two tiers, one harness, one benchmark and $T=4$ rounds.

5.2 Part C: Revision in Real Development Work

Data. 401 real sessions of AI-assisted software development at an enterprise software company. Unlike benchmark tasks, production work is not chosen to be nearly solvable, so it leaves the room for improvement the theory needs. We extract structural git metrics only, keeping no code, file paths, commit messages or author identities. A *session* is a run of consecutive AI-co-authored commits less than 4 hours apart: 401 sessions, 1,211 commits, 23 repositories. Construction, caveats on how model tiers are labeled, and all five findings are in Appendix F.

Finding 1: the amount of change falls off over a session. 70.0% of multi-commit sessions show *churn* (lines added plus lines deleted) decreasing from commit to commit, with a median late-to-early ratio of 0.49 (95% CI [0.400, 0.793]; Appendix Fig. 7, left). The order is genuinely temporal, not an artifact of which commits happen to be large: shuffling commit order destroys it (permutation test, $p<0.0001$, on the $n=70$ sessions with ≥ 5 commits; half the cohort is a single commit). Descriptively, then, current frozen-model agentic coding looks like the bounded regime (i): it behaves like an editor that converges, not a process that compounds.

Findings 2–3: what this does not show. It does not show the mechanism, for two reasons. First, the control that could have pointed to something AI-specific does not: 9,395 sessions by human programmers before AI tools show the same *shape* of decay (geometric decay factor $\rho_{\text{human}}=0.86$ against $\rho\approx 0.77$ for AI sessions). Fast early change followed by polishing is how editing works in general, and a fixed reachable set is not needed to predict it. Second, the one test specific to our theory comes back null. If a more capable model can reach more, its sessions should keep more of what they write; but Opus sessions show no higher *survivor ratio* ψ (net lines over total churn) than Sonnet sessions ($\psi_{\text{Opus}}\approx 0.55$, $n=158$ vs. $\psi_{\text{Sonnet}}\approx 0.55$, $n=177$; gap= -0.003 , 95% CI $[-0.075, +0.069]$, permutation $p=0.93$; Appendix Fig. 7, right). The null survives four checks for confounds and a re-test by model version. Since Part B finds a large capability effect, we read this as a limit of the proxy rather than of the models (§6), and as a *bounded* null, not a demonstrated zero.

6 Discussion

For oversight: what to monitor. Frontier safety frameworks already test models with strong scaffolding, so the gap is not in how capability is elicited but in *when* re-assessment happens: it is largely triggered by training-side events such as a finished post-training run or a jump in effective compute [Anthropic, 2026, Google DeepMind, 2026, OpenAI, 2025], and the frameworks themselves treat scaffold-driven gains as an open problem (Appendix G.4). Under (M1), a scaffold rewritten *between* assessments can grow what a system reaches with no training event to prompt a new look. The quantity to monitor is therefore whether the scaffold is expanding itself, not only whether the weights changed. A loop that only re-samples is bounded by construction; one that rewrites its own tools or task breakdown is not (Cor. 4). Self-expansion is also a reward-hacking risk, since a system allowed to rewrite its own tests can game them rather than improve (Appendix G.4–G.5).

For practitioners. Two lessons follow, with five in Appendix G.3. **Spend on the first attempt, not the revision loop:** later rounds mostly leave the code unchanged (Appendix Fig. 6). **Upgrade the model before adding orchestration:** orchestration cannot exceed the best worker’s ceiling (Prop. 5), and model tier outweighed every other factor we varied (partial $\eta^2=0.497$).

Why model strength matters in the lab but not in production. Part B’s tier contrast explains roughly half the variance; Part C’s finds no gap between tiers. We do not resolve this, but the two contrasts differ in ways that could each explain it. In production, developers choose the model rather than having it assigned at random, and Opus is often used for planning while Sonnet does the execution, so a capability gap can be offset by harder or different work going to the stronger model. Part C’s adjustments for task type, repository and developer remove the coarsest form of that selection, not selection within a task type. The outcomes differ too: Part B scores whether the tests pass on a fixed task, Part C a structural proxy on whatever work a session took on. And the models differ: Part B compares Sonnet with Haiku, Part C two upper tiers. Random model assignment on matched tasks, or a functional outcome in place of the proxy, would separate these explanations; this data does neither.

The decisive experiment, proposed and not run. The confounds above, and Part C’s human baseline, all point to one design this paper does not execute. Hold the model fixed and climb a ladder of scaffolds: (S0) no tools; (S1) the agent can run tests; (S2) it can also search or retrieve code; (S3) it can write tools during a run and keep them for later rounds. Run every rung on a fixed set of tasks with every round executed, including after a first pass, so neither confound of §5.1 applies. Record two curves per rung: the gain per round $\bar{\eta}_t$, and *pass@k coverage*, the share of tasks on which *any* of k attempts passes the tests [Brown et al., 2024]. Coverage is our *proposed* practical stand-in for the reachable set, which we otherwise cannot measure: comparing rungs at the same k asks whether a rung reaches tasks the previous one could not, rather than the same tasks sooner. It is a lower bound under a finite budget and depends on the tests, so it approximates the set rather than measuring it. The hypothesis predicts that within each fixed rung (S0–S2) gains shrink and coverage plateaus; that rungs adding new reach raise the coverage plateau with the model unchanged, which is (M1); and that only (S3), whose reach can grow mid-run, can show a plateau that rises within a run. It is falsified if coverage is flat across rungs, so these scaffolds do not expand reach and a model-level audit misses nothing on them, or if every rung decays like the human baseline whatever its reach, so the ordinary economics of editing explain the data.

Open problems. Each is falsifiable, and blocked by something we did not do rather than something we found. (1) **Build the harness** of §3, keeping every attempt, focusing on failing specifications, and enforcing $m^* < k/2$: does it improve at the rate boosting promises, or does a new gap appear only in practice? (2) **Run the vote across model vendors:** do workers from genuinely different families satisfy Prop. 6? The measurement is cheap and has not been made. If they fail it, voting orchestration has no viable regime on real code. (3) **Keep revising after the first pass:** one re-run that does not stop on success would give the unconditional per-round curve this data cannot. (4) **Hold the model fixed and vary the scaffold,** the design above, and the only one that separates a fixed reachable set from the ordinary economics of editing.

6.1 Limitations

The four below could each overturn a claim we make, or are why we do not make one; seven more that bound scope without threatening a specific result are in Appendix G.6.

- **The slowdown is not specific to AI, which is why the evidence is descriptive.** Human programmers before AI tools show the same shape of decay (§5.2). The data therefore place current practice in the bounded regime, but a fixed reachable set is not needed to predict that shape, so they cannot attribute it to one. Only a fixed-model design such as the one in §6 could.
- **The per-round measurements are conditional.** Because the harness stops calling the model once a task passes, the later rounds that would give an unconditional per-round curve were never run (§5.1). What we report conditions on a task still being unsolved, so the population shrinks and gets harder with each round (Sonnet 113→49 of 275), and part of any decline is a change of sample rather than of behavior. That applies to both the improvement edge and the gain per round; a fixed set of tasks with every round executed would remove it (open problem 3). Reading the strong model’s zero-crossing intervals as proof that revision *does* keep paying would over-read them as much as the pooled series over-reads the opposite. Part C does not measure the edge at all, so Cor. 11 is conditional throughout.
- **Rounds may restart rather than revise (an unchecked assumption).** The model treats each round as an edit of the previous draft, $z_t = R_t(z_{t-1}, a_t)$, but an agent can abandon its approach mid-task and start over. Thm. 2’s bound on the best score so far still holds for such a path; what fails is reading the shrinking gains as revision saturating, and reading patches as building on each other. Pass/fail scoring cannot tell a restart from a revision, and we did not check, for instance by comparing patches across rounds. Part B’s independent arm restarts by design, and the underpowered feedback null cannot separate it from the others.
- **Observational data, model choice, and version pooling.** In Part C the model tier is *not* randomly assigned: Opus is often used for planning and Sonnet for execution, so the tier contrast is partly a task-type contrast. The survivor ratio measures structural convergence, not whether the code works. Finding 2’s checks adjust for task type, repository and developer but not model version, which is why it is re-run by version. That the null survives rules out version pooling as the explanation without making it a demonstrated zero, and reading it as evidence *against* a capability effect would over-read it. Finding 3’s human comparison is likewise a comparison across eras.

Orchestrator-worker topologies are treated formally in Appendix C.

7 Conclusion

Viewing agentic coding through boosting yields a simple criterion: repeated self-revision levels off whenever the set of reachable edits is fixed and quality has a ceiling, and can escape only if that set keeps growing (Prop. 3). If scaffold rewrites can grow it with the model frozen (M1), a frozen model does not by itself guarantee a safe, levelling-off trajectory: audit the scaffold, not only the checkpoint. Adding agents does not change this. Picking the best answer reaches the best worker’s ceiling exactly (Prop. 5), a vote would need workers who fail on different tasks and ours do not (a majority fails 23/55 (42%) of tasks), and asking the same pool again has a limit computable in advance unless the pool can already fit the work (Prop. 7). Real loops depart from boosting by overwriting drafts and by failing together; both are design choices, and together they describe a harness worth building. What we observe is levelling off, on SWE-bench and across 401 production sessions, matched in shape by human programmers before AI tools. What remains open is why: a genuinely fixed reachable set, or the ordinary economics of editing. Holding the model fixed while varying the scaffold would tell them apart. This paper specifies that experiment; it does not run it.

Responsible Use Statement

Recursive self-improvement in coding agents is dual-use adjacent, but this paper’s finding runs the other way: a *saturation and limits* result, not an acceleration recipe. The criterion, whether the reachable hypothesis class is stationary, lets evaluators *recognize* when a self-modifying system could escape diminishing returns instead of reading frozen weights as assurance, and we release no

technique or scaffold that expands an agent’s reachable class. Part C releases four session datasets covering 71 people’s commit timing, structural only: developer identity is a salted hash whose roster is withheld, and no code content, file paths or commit messages survive (sanitization audit in Appendix F). This is IP-safe but not anonymous in the strong sense, since commit timing is in principle linkable by anyone holding the original repositories. We also disclose substantial AI assistance in drafting this manuscript.

Acknowledgments and Disclosure of Funding

The authors thank Tom Edwards and colleagues at OnCorps for feedback and discussions that improved this work. We gratefully acknowledge the engineers at OnCorps, past and present, whose production code forms the Part C dataset. We thank Brett Roads for detailed comments on an earlier draft that substantially improved its clarity and framing. We also thank Peter Riefer, Aristeidis Panos, Bradley Love and Simon Busch-Moreno for their feedback, and the Data Science Team at OnCorps for comments on the manuscript.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026.
- AI Security Institute, UK. Inspect: An open-source framework for large language model evaluations. <https://inspect.aisi.org.uk/>, 2024. Released 2024-05-10 as a platform of the UK AI Safety Institute, since renamed the AI Security Institute; developed with Meridian Labs. Framework source at github.com/UKGovernmentBEIS/inspect.ai, companion evaluation collection at github.com/UKGovernmentBEIS/inspect.evals. Accessed 2026-08-21.
- Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for few-shot learning. In *International Conference on Machine Learning (ICML)*, 2025.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016.
- Anthropic. Anthropic’s responsible scaling policy, version 3.4. <https://www.anthropic.com/rsp>, 2026. Effective 2026-07-08. Accessed 2026-08-18.
- Alina Beygelzimer, Satyen Kale, and Haipeng Luo. Optimal and adaptive algorithms for online boosting. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2323–2331, 2015.
- Nick Bostrom. *Superintelligence: Paths, dangers, strategies*. Oxford University Press, 2014.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024.
- Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in ai: From bounded self-refinement to autonomous research loops, 2026.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.
- Marina Favaro and Jack Clark. When ai builds itself: Our progress toward recursive self-improvement, and its implications. The Anthropic Institute, <https://www.anthropic.com/institute/recursive-self-improvement>, 2026. Accessed 2026-08-03.
- Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Additive logistic regression: A statistical view of boosting. *Annals of Statistics*, 28(2):337–407, 2000.
- Jerome H Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

- Irving John Good. Speculations concerning the first ultraintelligent machine. *Advances in computers*, 6:31–88, 1965.
- Google DeepMind. Frontier safety framework, version 3.1. <https://deepmind.google/discover/blog/strengthening-our-frontier-safety-framework/>, 2026. Effective 2026-04-17. Accessed 2026-08-18.
- Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Chaoyue He, Xin Zhou, Di Wang, Hong Xu, Wei Liu, and Chunyan Miao. Harness engineering for language agents: The harness layer as control, agency, and runtime. Preprints.org, 2026.
- Danny Hernandez and Tom B Brown. Measuring the algorithmic efficiency of neural networks, 2020.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework, 2023.
- Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation, 2023.
- Jonas Hübötter, Sascha Bongni, Ido Hakimi, and Andreas Krause. Efficiently learning at test-time: Active fine-tuning of LLMs. In *International Conference on Learning Representations (ICLR)*, 2025.
- Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion, 2023.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- Subbarao Kambhampati. Can large language models reason and plan?, 2024.
- Ara Khan. Recursive self-improvement for coding agents. Cline Blog, <https://cline.bot/blog/recursive-self-improvement-for-coding-agents>, 2026. Accessed 2026-08-03.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback, 2023.
- Llew Mason, Jonathan Baxter, Peter Bartlett, and Marcus Frean. Boosting algorithms as gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 12. MIT Press, 1999.
- J. Pablo Muñoz and Jinjie Yuan. RTTC: Reward-guided collaborative test-time compute, 2025.
- Nachiappan Nagappan and Thomas Ball. Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th International Conference on Software Engineering (ICSE)*, pages 284–292, 2005.
- OpenAI. Preparedness framework, version 2. <https://cdn.openai.com/pdf/18a02b5d-6b67-4cec-ab64-68cdfbdebcd/preparedness-framework-v2.pdf>, 2025. Last updated 2025-04-15. Accessed 2026-08-18.
- Kartik Ghanshyambhai Pansuriya, Ehsan Ghorbani, Deepak Singh, and Eman Abdullah AlOmar. Predicting acceptance and review effort in human and agent pull requests, 2026.
- Theodore Papamarkou, Vladislav Smirnov, Viktor Mazanov, Artem Vazhentsev, Preslav Nakov, Timothy Baldwin, and Artem Shelmanov. Bayesian control for coding agents, 2026.

- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In *ACL 2024*, 2024.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- Robert E Schapire. Explaining adaboost. *Empirical Inference*, pages 37–52, 2013.
- Robert E Schapire and Yoram Singer. Improved boosting algorithms using confidence-rated predictions. *Machine Learning*, 37(3):297–336, 1999. doi: 10.1023/A:1007614523901.
- Robert E Schapire, Yoav Freund, Peter Bartlett, and Wee Sun Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. *Annals of Statistics*, 26(5):1651–1686, 1998.
- Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements, 2003.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024.
- Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning (ICML)*, 2020.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *International Conference on Machine Learning (ICML)*, 2025.
- Ioannis Tschantzaris, Thorsten Joachims, Thomas Hofmann, and Yasemin Altun. Large margin methods for structured and interdependent output variables. *Journal of Machine Learning Research*, 6:1453–1484, 2005.
- Tyler J. VanderWeele and Peng Ding. Sensitivity analysis in observational research: Introducing the e-value. *Annals of Internal Medicine*, 167(4):268–274, 2017. doi: 10.7326/M16-2607.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities, 2024.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022.
- Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct, 2023.
- Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models, 2024.

- Hui Xue and Fan Yang. Rethinking self-evolving agents: Do we still need prescribed optimization pipelines?, 2026.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation, 2023.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents, 2025a.
- Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. A survey on test-time scaling in large language models: What, how, where, and how well?, 2025b.
- Ji Zhu, Hui Zou, Saharon Rosset, and Trevor Hastie. Multi-class adaboost. *Statistics and Its Interface*, 2(3):349–360, 2009.
- Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

A The Boosting/Coding Mapping, and the Protocol

Table 1 gives the term-by-term mapping that §3 states in prose; Algorithm 1 below is the Protocol whose exact form §3 reads as a build specification rather than a description of any deployed loop.

Boosting concept	Agentic coding analog	Concrete example
Input X	Specification	Prompts, GitHub tickets
Target Y	Optimal Code	The optimal “best” code for X
Current State $F_{t-1}(X)$	Current Draft	The code generated by previous agents
Residual $(Y - F_{t-1}(X))$	Code Edit	A git diff or patch
Weak learner h_t	Agent Patch Generator	LLM call proposing an edit
Loss Function $L(Y, \hat{Y})$	Structured Distance	Token edit distance, AST distance
Distribution D_t	Spec Weighting	Focus on failing specifications

Table 1: Mapping between gradient boosting and agentic coding.

A.1 Algorithm: Agentic Boosting Protocol

Algorithm 1 Agentic Boosting Protocol for Specifications

Require: Spec-Code pairs $\{(x_i, y_i)\}_{i=1}^N$, number of rounds T

- 1: Initialize $w_i \leftarrow 1/N$ for all i
- 2: **for** $t = 1$ to T **do**
- 3: Select weak coding agent h_t minimizing $\epsilon_t = \sum_i w_i \mathbf{1}[h_t(x_i) \neq y_i]$
- 4: **if** $\epsilon_t \geq 1/2$ **then**
- 5: Stop: no weak learner available
- 6: **end if**
- 7: Set $\alpha_t \leftarrow \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$
- 8: Update $w_i \leftarrow w_i \cdot \exp(-\alpha_t y_i h_t(x_i))$ for all i
- 9: Normalize $w_i \leftarrow w_i / \sum_j w_j$
- 10: **end for**
- 11: **return** artifact $H(x) = (R_T \circ \dots \circ R_1)(z_0)$ (sequential patch composition), with classification confidence given by the additive margin $f(x) = \sum_t \alpha_t h_t(x)$

B Full Proofs of the Theoretical Results

Note on formalization scope and motivation: the algebraic steps in this section have been machine-checked in Lean 4 (Mathlib) as a quality-control layer against AI-hallucinated derivations, since this manuscript was substantially drafted with AI assistance. The result-to-declaration map is in Appendix H; what that index cannot do is certify the modelling assumptions, which Remark 16 states plainly.

B.1 Statements of the Transferred Classical Results

Summarized in §4 and stated formally here, next to their proofs; the main text relies on none of them. What they require is not a positive edge—§3 withdraws that reading—but a *retained, reweighted ensemble*: a distribution D_t maintained over specifications, and all T hypotheses kept and combined by weighted vote rather than overwritten. No deployed refinement loop does either. Three transfer verbatim under this reading and none is new: the training-error product bound (Thm. 10) and its $e^{-2\gamma^2 T}$ corollary under a uniform edge (Cor. 11); equivalence to forward stagewise additive modeling under exponential loss (Thm. 13); and Schapire et al. [1998]’s margin bound, making generalization depend on weak-learner complexity rather than round count (Thm. 12)—though for an LLM patch generator the VC-dimension is unknown, examples need not be i.i.d., and a self-expanding scaffold breaks the fixed- $\mathcal{H}$ premise outright.

Definition 8 (Strong Coding System). *A strong coding system is, at the artifact level, the sequential composition $F_T(x) = (R_T \circ \dots \circ R_1)(F_0(x))$ of patches (order-dependent, non-linear); at the score*

level, the additive margin $f(x) = \sum_{t=1}^T \alpha_t h_t(x)$, $\alpha_t \geq 0$. It is this margin, not the code bytes, that inherits the AdaBoost machinery—which is exactly what a composing loop does not build.

Definition 9 (Agentic Boosting Protocol). *The Agentic Boosting Protocol (Algorithm 1) maintains a distribution D_t over $\{(x_i, y_i)\}$; at round t it selects h_t minimizing $\epsilon_t = \Pr_{D_t}[h_t(x) \neq y]$, sets $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$, and updates $D_{t+1}(i) \propto D_t(i) \exp(-\alpha_t y_i h_t(x_i))$.*

Theorem 10 (Training Error of Agentic Boosting). *After T rounds of the Agentic Boosting Protocol, the training error of the strong coding system H is bounded by $\frac{1}{N} \sum_i \mathbf{1}[H(x_i) \neq y_i] \leq \prod_{t=1}^T Z_t$, where $Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)}$.*

Corollary 11 (Exponential Decay of Training Error). *If each weak coding agent has edge $\gamma_t \geq \gamma > 0$ for all rounds t , then $\text{err}_{\text{train}}(H) \leq \exp(-2\gamma^2 T)$.*

Theorem 12 (Generalization Error of Agentic Boosting). *Let $\mathcal{H}$ be the hypothesis class of weak coding agents with VC-dimension d , and let the margin be measured on the normalized combination $f / \sum_t \alpha_t$, so $yf(x) \in [-1, 1]$ and the threshold is $\theta \in (0, 1]$. Running the Protocol for T rounds on N i.i.d. samples, with probability at least $1 - \delta$: $\Pr_{\mathcal{D}}[H(x) \neq y] \leq \Pr_{\text{train}}[yf(x) \leq \theta] + O\left(\sqrt{\frac{1}{N} \left(\frac{d \ln^2(N/d)}{\theta^2} + \ln \frac{1}{\delta}\right)}\right)$, where the second term is independent of T [Schapire et al., 1998].*

For a real LLM patch generator, d is typically unknown, examples need not be i.i.d., and a self-expanding scaffold can violate the fixed- $\mathcal{H}$ assumption (§G.6).

Theorem 13 (Boosting as Greedy Coordinate Descent). *The Agentic Boosting Protocol is equivalent to forward stagewise additive modeling with exponential loss $L(y, f(x)) = e^{-yf(x)}$: at each round t it greedily selects the weak agent and weight that most reduce $J(\alpha, h) = \sum_i \exp(-y_i(f_{t-1}(x_i) + \alpha h(x_i)))$. This is Friedman et al. [2000]’s statistical view of boosting, read over specifications.*

B.2 Proofs

Grouped by family: first the four transferred classical results, which we rely on nowhere; then the refinement game and the dichotomy, which carry the paper; then the frozen-weight ceiling the dichotomy instantiates.

Transferred classical results.

Proof of Theorem 10. Let $f(x_i) = \sum_{t=1}^T \alpha_t h_t(x_i)$ be the unnormalized weighted sum. The training error can be bounded via the exponential loss:

$$\frac{1}{N} \sum_{i=1}^N \mathbf{1}[H(x_i) \neq y_i] \leq \frac{1}{N} \sum_{i=1}^N \exp(-y_i f(x_i)).$$

We prove this by induction on T . For $T = 0$, $f \equiv 0$, so the bound gives 1, which is trivially satisfied.

Inductive step. Assume the bound holds after $T - 1$ rounds with weights $D_T(i) = \frac{\exp(-y_i \sum_{t=1}^{T-1} \alpha_t h_t(x_i))}{N \prod_{s=1}^{T-1} Z_s}$. After round T , we have:

$$\begin{aligned} \frac{1}{N} \sum_{i=1}^N \exp\left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i)\right) &= \sum_{i=1}^N D_T(i) \cdot \prod_{t=1}^{T-1} Z_t \cdot \exp(-y_i \alpha_T h_T(x_i)) \\ &= \prod_{t=1}^{T-1} Z_t \sum_{i=1}^N D_T(i) \exp(-y_i \alpha_T h_T(x_i)). \end{aligned}$$

We split the sum into correctly and incorrectly classified samples:

$$\begin{aligned} \sum_{i=1}^N D_T(i) \exp(-y_i \alpha_T h_T(x_i)) &= \sum_{i: h_T(x_i)=y_i} D_T(i) e^{-\alpha_T} + \sum_{i: h_T(x_i) \neq y_i} D_T(i) e^{\alpha_T} \\ &= (1 - \epsilon_T) e^{-\alpha_T} + \epsilon_T e^{\alpha_T}. \end{aligned}$$

Substituting $\alpha_T = \frac{1}{2} \ln \frac{1-\epsilon_T}{\epsilon_T}$:

$$\begin{aligned} (1 - \epsilon_T)e^{-\alpha_T} + \epsilon_T e^{\alpha_T} &= (1 - \epsilon_T)\sqrt{\frac{\epsilon_T}{1 - \epsilon_T}} + \epsilon_T\sqrt{\frac{1 - \epsilon_T}{\epsilon_T}} \\ &= \sqrt{\epsilon_T(1 - \epsilon_T)} + \sqrt{\epsilon_T(1 - \epsilon_T)} \\ &= 2\sqrt{\epsilon_T(1 - \epsilon_T)} = Z_T. \end{aligned}$$

Combining:

$$\frac{1}{N} \sum_{i=1}^N \exp(-y_i f(x_i)) = \prod_{t=1}^{T-1} Z_t \cdot Z_T = \prod_{t=1}^T Z_t.$$

Since $1[H(x_i) \neq y_i] \leq \exp(-y_i f(x_i))$ (because $\text{sign}(f(x_i)) \neq y_i \implies y_i f(x_i) \leq 0 \implies \exp(-y_i f(x_i)) \geq 1$), the training error is bounded by $\prod_{t=1}^T Z_t$. ■

Proof of Corollary 11. From Theorem 10, $\text{err}_{\text{train}} \leq \prod_{t=1}^T Z_t$ where $Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)}$. Since $\epsilon_t \leq \frac{1}{2} - \gamma$, we have:

$$\begin{aligned} Z_t &= 2\sqrt{\epsilon_t(1 - \epsilon_t)} \\ &\leq 2\sqrt{\left(\frac{1}{2} - \gamma\right)\left(\frac{1}{2} + \gamma\right)} \\ &= 2\sqrt{\frac{1}{4} - \gamma^2} \\ &= \sqrt{1 - 4\gamma^2}. \end{aligned}$$

Using the inequality $\sqrt{1 - u} \leq e^{-u/2}$ for $u \in [0, 1]$:

$$\prod_{t=1}^T Z_t \leq \left(\sqrt{1 - 4\gamma^2}\right)^T \leq \left(e^{-2\gamma^2}\right)^T = e^{-2\gamma^2 T}.$$

As $T \rightarrow \infty$, the training error converges to zero exponentially fast. ■

Remark 14 (The edge condition is a capability threshold, not a weak requirement). The hypothesis $\gamma_t \geq \gamma > 0$ is substantially more demanding in code space than the analogous condition in ordinary binary classification, because the space is overwhelmingly hostile: almost all edits to a working program break it, so an error below $\frac{1}{2}$ already presupposes a considerable baseline of program understanding. That baseline is what Kambhampati [2024] argues single-model reasoning does not reliably supply, which is one reason we treat a positive edge as a hypothesis to be measured rather than granted. That is a statement about the *edit space*, not about the agents we measure; read as a claim about them it would predict the wrong thing.

Deployed patches are not destructive: on the diagnostic arm at most 2.9% of rounds lower quality, and 80.7%/69.1% of trajectories never change it at all (Appendix E.1). The measured shortfall is idempotence, not damage, which is why a negative pooled γ here cannot be read as these agents being anti-correlated with correctness. Corollary 11 therefore guarantees exponential convergence only *once* the base agents cross this capability threshold. Agents with weighted error $\epsilon_t \geq \frac{1}{2}$ (non-positive edge) receive weight $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t} \leq 0$ and are gated out by the stopping rule of Algorithm 1; the protocol thus performs a form of capability warm-up, admitting an agent to the ensemble only after it becomes a genuine weak learner.

Proof sketch of Theorem 12. Normalizing by $\sum_t \alpha_t$ makes H a convex combination of T weak learners from $\mathcal{H}$, which is the form the margin bound requires; $H = \text{sign}(f)$ is unchanged by the rescaling. By the margin bound of Schapire et al. [1998], for any $\theta \in (0, 1]$ and with probability $1 - \delta$:

$$\Pr_{\mathcal{D}}[y f(x) \leq 0] \leq \Pr_{\text{train}}[y f(x) \leq \theta] + O\left(\sqrt{\frac{1}{N} \left(\frac{d \ln^2(N/d)}{\theta^2} + \ln \frac{1}{\delta}\right)}\right).$$

The right-hand side does not depend on T ; it is controlled by the VC-dimension d of the weak-learner class and the margin threshold θ , and it degrades as θ^{-2} , so a bound quoted with a

weaker θ -dependence would overstate it. As boosting increases margins on training data, the term $\Pr_{\text{train}}[y f(x) \leq \theta]$ decreases, giving better generalization. Balancing the two terms in θ would give an optimal stopping time of order $\frac{1}{\gamma^2} \ln \frac{N}{d}$; we do not derive it, and Cor. 11 alone does not supply it—that corollary bounds the *zero-one* training error, whereas the balance needs the θ -margin training bound, a different inequality [Schapire et al., 1998]. Nothing in this paper uses the stopping time. ■

Proof of Theorem 13. Fix round t and let $w_i^{(t)} = \exp(-y_i f_{t-1}(x_i))$ be the unnormalized weights. The objective is:

$$J(\alpha, h) = \sum_{i=1}^N w_i^{(t)} \exp(-y_i \alpha h(x_i)).$$

Splitting by correctness:

$$J(\alpha, h) = e^{-\alpha} \sum_{i: h(x_i)=y_i} w_i^{(t)} + e^{\alpha} \sum_{i: h(x_i) \neq y_i} w_i^{(t)}.$$

In terms of the weighted error $\epsilon = \frac{\sum_{i: h(x_i) \neq y_i} w_i}{\sum_i w_i}$:

$$J(\alpha, h) = \left(\sum_i w_i \right) [(1 - \epsilon)e^{-\alpha} + \epsilon e^{\alpha}].$$

The factor $\sum_i w_i$ does not depend on α or h . Minimizing over α for fixed h (setting $\frac{\partial J}{\partial \alpha} = 0$):

$$-(1 - \epsilon)e^{-\alpha} + \epsilon e^{\alpha} = 0 \implies \alpha^* = \frac{1}{2} \ln \frac{1 - \epsilon}{\epsilon},$$

which is exactly the AdaBoost weight update rule. Minimizing over h for fixed $\alpha > 0$ amounts to minimizing ϵ , i.e., choosing the agent with the lowest weighted error. This establishes the equivalence between the Agentic Boosting Protocol and greedy coordinate descent on the exponential loss. ■

The refinement game and the dichotomy. These are the results the paper relies on, and they mention AdaBoost nowhere.

Proof of Theorem 2. By telescoping:

$$\mathbb{E}[V(z_T)] - V(z_0) = \sum_{t=1}^T (\mathbb{E}[V(z_t)] - \mathbb{E}[V(z_{t-1})]) \geq \sum_{t=1}^T \eta_t.$$

Since $V \leq 1$, we have $\mathbb{E}[V(z_T)] \leq 1$, so $\sum_{t=1}^T \eta_t \leq 1 - V(z_0)$ for all T , which implies $\sum_{t=1}^{\infty} \eta_t \leq 1 - V(z_0)$.

Connection to boosting. The quality function V plays the role of the margin $yf(x)$, the quantity boosting drives upward. The refinement property is the analog of the weak learning condition (Definition 1): each agent must improve quality by at least η_t , just as each weak learner must have edge at least γ_t . The saturation bound is the analog of Corollary 11 only in its *consequence*—the number of useful rounds is finite—and not in its form: Cor. 11 is a rate, whereas $\sum_t \eta_t \leq 1 - V(x_0)$ bounds the total improvement without asserting how it is distributed over rounds. No rate is claimed here (§1). ■

The high-probability form of the refinement game trades the improvement property’s expectation for a per-round tail bound. The main text relies on it nowhere.

Corollary 15 (High-Probability Refinement). *In the setting of Thm. 2, suppose instead $\Pr[V(z_t) - V(z_{t-1}) \geq \eta_t] \geq 1 - p_t$ for each round. Then $\Pr[V(z_T) \geq V(z_0) + \sum_{t=1}^T \eta_t] \geq 1 - \sum_{t=1}^T p_t$.*

Proof of Corollary 15. Let E_t be the event that $V(z_t) - V(z_{t-1}) \geq \eta_t$. By assumption, $\Pr[E_t] \geq 1 - p_t$. Using the union bound, the probability that all events $E_1, \dots, E_T$ occur simultaneously is at least $1 - \sum_{t=1}^T p_t$. When all E_t occur, the telescoping sum yields $V(z_T) - V(z_0) = \sum_{t=1}^T (V(z_t) - V(z_{t-1})) \geq \sum_{t=1}^T \eta_t$. This relaxes the strict monotone requirement to a high-probability guarantee, more accurately reflecting stochastic agentic code edits. ■

Proof of Proposition 3. Part (i) generalizes the saturation bound of Theorem 2 from the ceiling 1 to an arbitrary B : by monotonicity, $\mathbb{E}[V(z_T)] \leq B$, and combined with $\mathbb{E}[V(z_T)] \geq \mathbb{E}[V(z_0)] + \sum_{t=1}^T \eta_t$ we obtain $\sum_{t=1}^T \eta_t \leq B - \mathbb{E}[V(z_0)]$ for every T . For part (ii), the telescoping lower bound of Theorem 2 uses only per-round improvement and gives $\mathbb{E}[V(z_T)] \geq \mathbb{E}[V(z_0)] + \sum_{t=1}^T \eta_t \geq \mathbb{E}[V(z_0)] + Tc$. By the Archimedean property, for any B we may choose $T > (B - \mathbb{E}[V(z_0)])/c$, giving $\mathbb{E}[V(z_T)] > B$. ■

Part (i) is the *stationary* regime: when the class of reachable patches is fixed, a bounded quality metric mathematically enforces a plateau. Part (ii) is the *non-stationary* regime: sustaining a positive edge indefinitely requires the reachable class itself to keep growing, which is precisely what an unbounded “intelligence explosion” would demand.

The frozen-weight ceiling.

Proof of Corollary 4. Part (i) applies Proposition 3(i) with the fixed bound $B = V^*(W)$, whose existence is assumption (M2); monotone convergence of the bounded, non-decreasing sequence $\mathbb{E}[V(x_t)]$ gives the limit $L \leq V^*(W)$. Part (ii) is the contrapositive of Proposition 3(ii): if a fixed ceiling $V^*(W)$ bounds V , then no uniform edge $\eta_t \geq c > 0$ can be sustained, so $\mathbb{E}[V(x_T)]$ cannot exceed every bound. Part (i)’s *saturation bound*— $\sum_{t < T} \eta_t \leq V^*(W) - \mathbb{E}[V(x_0)]$ for every T —is machine-checked in Lean as `cor_frozen_weight_ceiling` (a direct instantiation of `prop8_saturation_forces_decay` with $B := V^*(W)$); the step from there to the limit L is the monotone-convergence argument above and is not formalized, since the Lean statement does not assume $\eta_t \geq 0$. Part (ii) is backed by `prop8_sustained_edge_diverges`. ■

What the corollary buys—that frozen weights secure the eventual ceiling but not stationary behaviour on the way to it, since (M1) lets $\mathcal{H}_t(C)$ expand while W is fixed—is the paper’s governance claim, and §G.4 develops it with the mechanisms that move a system between the two regimes.

Remark 16 (Lean verifies the algebra, not the modeling assumptions). Theorem 2 and Proposition 3 are elementary results: part (i) is a bounded-monotone-sequence argument; part (ii) is the Archimedean property. Lean verifies the formal algebra of these proofs—it does *not* certify the modeling assumptions (the per-round improvement property, (M1), (M2)) that connect the abstract game to real agentic systems. Those assumptions are stated as explicit hypotheses and remain empirical claims.

C Orchestration, Voting, and the Diversity a Vote Needs

The dismissal of “elaborate multi-agent orchestration” in the main text and §G.3 (item 3) is narrower than it may read: it says orchestration cannot raise any individual worker’s ceiling $V^*(W_i)$ —a stronger model still wins by clearing tasks no arrangement of weaker models can reach. It is not an argument that hierarchical or orchestrator-worker *topologies* fall outside the framework; the multi-agent code-generation systems surveyed in §G.1 already route work across specialized agents, and the paper’s own $\mathcal{S} = (W, C)$ framing (§G.4) already lists orchestration as part of the mutable scaffold C . We make that instance explicit.

Setup. Replace single-agent sequential refinement with k frozen workers under scaffolds $C_1, \dots, C_k$ (distinct prompts, tools, or specializations, possibly sharing weights W), coordinated by an orchestrator that, each round, selects among their outputs. Let $\mathcal{H}_t(C_i)$ be worker i ’s reachable class (Cor. 4’s framing), and define the orchestrator’s reachable class under best-of- k selection as $\mathcal{H}_t^{\text{orch}} := \bigcup_{i=1}^k \mathcal{H}_t(C_i)$.

C.1 Hard Selection Reaches the Best Worker, Exactly

Proof of Prop. 5 (stated in §4). For any instance x , the orchestrator’s best-of- k patch attains value $\max_i V(h_i(x))$ over each worker’s best reachable patch, so $V_{\text{orch}}^* = \sup_x \max_i V(h_i(x))$. Sup and finite max commute unconditionally: $\sup_x \max_i V(h_i(x)) = \max_i \sup_x V(h_i(x)) = \max_i V^*(W, C_i)$, with no side condition on the $\mathcal{H}_t(C_i)$ ’s. (One might expect a strict inequality whenever no single worker’s class contains the union, on the intuition that “the winning worker varies with x ” should cost something; it does not—two disjoint singleton classes $\mathcal{H}_t(C_1) = \{a\}$, $\mathcal{H}_t(C_2) = \{b\}$ with $V(a) = V(b)$ already give a counterexample, since neither singleton contains $\{a, b\}$ yet the ceilings tie exactly. Equality is the only case; there is no strict regime for hard selection.) Taking the union of k scaffold-indexed classes is itself an instance of assumption (M1): adding workers or routing among them is a scaffold rewrite (orchestration is already named as part of C , §G.4) that can expand $\mathcal{H}_t(C)$ without touching any W_i . Orchestrator-worker architectures do not escape Cor. 4; best-of- k selection instantiates its escape clause exactly, never more. This equality is machine-checked in Lean as `prop15_tight_ceiling_orchestration`; a separate, weaker fact (`prop15_orchestration_ceiling`, `prop15_orchestration_dominates`) covers the case where each $V^*(W, C_i)$ is only an upper-bound hypothesis rather than a tight supremum, giving $V_{\text{orch}}^* \geq \max_i V^*(W, C_i)$ in that weaker setting. ■

Relation to the main-text guidance. This sharpens, rather than merely complements, “reach for capability before orchestration” (§G.3, item 3): that guidance says orchestration cannot raise any $V^*(W, C_i)$ itself, and Prop. 5 now shows best-of- k orchestration cannot exceed the best individual worker *either*—the equality is exact, not merely an upper bound. Routing among a fixed set of frozen workers is bounded by, never exempt from, the frozen-weight ceiling; the only lever a scaffold rewrite of this kind provides is selecting which worker’s already-existing ceiling to realize, an instance of (M1)’s class-expansion mechanism, not a mechanism special to multi-agent systems.

C.2 Weighted Voting, and the Diversity It Requires

Weighted aggregation: a partial answer. Best-of- k hard selection reduces cleanly to a union of reachable classes, and Prop. 5 shows that union buys *exactly* the best individual worker’s ceiling—no more. The natural place to look for genuine synergy is *weighted or soft aggregation*: an orchestrator that votes or blends across workers rather than selecting one is structurally closer to AdaBoost’s margin $f = \sum_t \alpha_t h_t$ (§3). Unlike sequential patch refinement, this mapping is not merely *boosting-inspired*—it is *boosting-exact*: workers responding to the *same* instance stand beside each other to be voted on rather than composing (§G.2’s objection to treating decomposition as an implicit vote does not apply here, since nothing is generated on the fly). Re-reading the index t in Algorithm 1 as “worker” rather than “round” costs nothing—Thm. 10 and Cor. 11 transfer verbatim.

Proposition 17 (Weighted Aggregation Beats the Weak-Learning Baseline). *Let k frozen workers be combined via the AdaBoost weighted vote rather than best-of- k hard selection, incorporated in sequence $h_0, \dots, h_{k-1}$, each maintaining weighted error $\text{err}_t \leq \frac{1}{2} - \gamma$ against the reweighted distribution current when it is incorporated, for some fixed $\gamma \in (0, \frac{1}{2})$. Then there exists k_0 such that for all $k \geq k_0$, the ensemble’s zero-one training error is strictly below $\frac{1}{2} - \gamma$ —the weak-learning threshold any single worker is only guaranteed to satisfy, not guaranteed to beat.*

Proof sketch. The zero-one training error is at most $\exp(-2\gamma^2 k)$ (Thm. 10, Cor. 11), which $\rightarrow 0$ as $k \rightarrow \infty$ while $\frac{1}{2} - \gamma$ is a fixed positive constant; take k_0 large enough that $\exp(-2\gamma^2 k_0) < \frac{1}{2} - \gamma$, which exists since $\exp(-2\gamma^2 k)$ is eventually smaller than any fixed positive threshold. Machine-checked in Lean as `cor_soft_aggregation_beats_weak_baseline`, built on the pure-analysis fact `exists_ensemble_size_beating_weak_threshold`. ■

Width and depth together. Prop. 5 and Prop. 17 both concern a *single* decision: k workers, one selection or one vote. The refinement game (Thm. 2) concerns the opposite axis: one agent, T sequential rounds. Neither addresses their composition— k workers consulted on *every* one of T rounds—which is what an orchestrated loop actually runs. For hard selection the composition goes through cleanly.

Proposition 18 (Multi-Round Best-of- k Orchestration). *Run the refinement game with k frozen workers consulted every round, the orchestrator retaining whichever output has the highest quality,*

so that $V(R_t^{(i)}(x, a)) \leq V(R_t^{\text{orch}}(x, a))$ for every worker i (such an R^{orch} always exists). If worker i satisfies Thm. 2’s improvement property with edge $\eta_t^{(i)}$, then

$$\mathbb{E}[V(z_T)] \geq V(z_0) + \sum_{t=1}^T \max_i \eta_t^{(i)} \geq V(z_0) + \sum_{t=1}^T \eta_t^{(j)} \quad \text{for every worker } j,$$

and, since $V \leq 1$, the total improvement still saturates: $\sum_{t=1}^T \max_i \eta_t^{(i)} \leq 1 - V(z_0)$. The main text states the same result compactly (Prop. 5).

Proof sketch. Fix a round t and let i^* attain $\max_i \eta_t^{(i)}$. Hard selection dominates worker i^* pointwise, so the orchestrated round inherits worker i^* ’s improvement and the orchestrated per-round edge is $\max_i \eta_t^{(i)}$; Thm. 2’s telescoping applies verbatim to that edge, and its saturation argument to the same sum. The middle inequality is $\eta_t^{(j)} \leq \max_i \eta_t^{(i)}$ summed over rounds. Machine-checked in Lean as `thm_width_refinement_game`, `cor_width_dominates_best_worker`, and `cor_width_saturation`, with `exists_width_orchestrator` discharging the non-vacuity of the domination hypothesis. One hypothesis beyond Thm. 2’s list is needed and is flagged as such in the formalization: monotonicity of the *inner* action-expectation, not only the outer one, since the selection step compares two integrands under the same expectation. ■

Width buys rate, not budget. The two inequalities pull in opposite directions. The first says orchestration strictly helps per round: the cumulative guarantee tracks the *best* worker available at each round and dominates every individual worker’s depth-only guarantee—not merely their average, and without knowing in advance which worker will win. The second says this cannot compound: the total improvement budget $1 - V(z_0)$ is fixed by the bounded metric, so consulting k workers every round spends that budget faster rather than enlarging it. Depth×width composition does not escape the frozen-weight ceiling (Cor. 4); it reaches it sooner. This is the multi-round counterpart of Prop. 5’s single-decision equality, and it extends “reach for capability before orchestration” (§G.3, item 3) into the time dimension: adding rounds to an orchestrated loop is subject to the same saturation as adding rounds to a single agent.

Quantifying the diversity a vote needs. Prop. 17’s edge hypothesis is qualitative and, more awkwardly, is stated against the *reweighted* distribution D_t , so it cannot be checked from primitive facts about the workers. Prop. 6 (stated in §4) gives a deterministic route instead, carrying diversity in a single integer. One consequence it leaves implicit is the form a designer would actually use: if any m bounds every m_i and $m < k/2$, the vote has zero training error (`thm_bounded_overlap_zero_error`), so a pool can be certified from an upper bound on overlap without computing m^* exactly.

Proof sketch of Prop. 6 (stated in §4). Splitting the margin at point i over the workers that get i right and those that get it wrong gives the identity $y_i f(x_i) = k - 2m_i$. The zero-one loss is an average of nonnegative indicators, so it vanishes exactly when every indicator does, i.e. when $y_i f(x_i) > 0$ for every i , i.e. when $k - 2m_i > 0$ —the claimed equivalence (`thm_overlap_iff_zero_error`, with `cor_tight_overlap_iff` for the m^* form), from which the sufficient direction above follows in one line. For sharpness, a single point at $2m_i = k$ has margin exactly 0—a tie, which the zero-one loss counts as an error, as it must, since a tied vote carries no information—so $2m < k$ cannot be weakened to $2m \leq k$. If every point sits at $m_i = k/2$, every margin vanishes and the error is 1; that uniform hypothesis is what `thm_overlap_boundary_sharp` assumes, and the even split witnesses that the case is non-vacuous (`exists_boundary_overlap_family`). The tight overlap reaching $k/2$ on its own gives only the first of the two, since points below the max keep positive margins. The unweighted vote is the existing margin $f = \sum_t \alpha_t h_t$ at $\alpha_t \equiv 1$, so no separate notion of voting is introduced. ■

Overlap is a real constraint, and counting alone buys nothing. First, at the tight overlap the condition is *exact*, not merely sufficient (Prop. 6), so it cannot be traded for a weaker one: it is a real constraint on orchestration design rather than a formality. Workers drawn from genuinely different model families, scaffolds, or prompts plausibly satisfy it, whereas the same model resampled does not—its failures concentrate on the same hard points, driving m toward k . That second half is

measurable on Part B rather than merely plausible, and we measure it below. Second, dropping the condition and counting alone buys *nothing*: double-counting failures and applying Markov gives only that the vote’s zero-one training error is at most $2\bar{\epsilon}$, twice the average per-worker error under the uniform distribution (`cor_overlap_markov_bound`), and for $\bar{\epsilon} < \frac{1}{2}$ the bound $2\bar{\epsilon}$ is *weaker* than $\bar{\epsilon}$ —weaker than using a single worker. Nor could a counting-only bound do better: the adversarial configuration in which every worker fails on the same $\bar{\epsilon}N$ points genuinely has ensemble error $\bar{\epsilon}$. It is the overlap hypothesis, not individual accuracy, that does the work—the precise sense in which *diversity* is what the vote needs.

Measuring the overlap: same-family workers fail together. Prop. 6 is checkable from primitive facts about the workers, which means it can be checked against data rather than asserted. Part B supplies 30 workers on a shared task pool: the 6 arms (two capability tiers $\times$ three feedback modes) $\times$ 5 seeds, all evaluated on the same 55 SWE-bench instances. Reading a worker’s ± 1 output as whether it solves a task—running-best quality reaching τ , the same criterion used throughout §5.1—gives m_i directly. The result is the extreme of the pessimistic case: the tight overlap is $m^*=30=k$, not merely above the $k/2=15$ threshold but at its ceiling, and it is $m^*=k$ at *every* round, not only the last. Some task is failed by all 30 workers; 23/55 (42%) of tasks have $2m_i \geq k$, which by Prop. 6 is the unweighted vote’s zero-one error (95% CI [0.291, 0.545], cluster bootstrap over tasks as in §5.1). Fig. 1 shows the distribution: a mode of easy tasks no worker fails, a second mode straddling $k/2$, and a spike at $m_i=k$.

One decisive statistic, and several weaker ones. Decisive: $m^*=k$. No interval is needed for a statistic sitting at its maximum, and it is the overlap condition, not any error comparison, that Prop. 6 makes exact. Weaker: everything involving $\bar{\epsilon}$. The mean per-worker error is $\bar{\epsilon}=0.356$ (CI [0.277, 0.436]), so the Markov bound $2\bar{\epsilon}=0.712$ is satisfied but nearly vacuous—indeed weaker than $\bar{\epsilon}$ itself, exactly as predicted for $\bar{\epsilon} < \frac{1}{2}$. The vote’s error 0.418 does sit above $\bar{\epsilon}$ at every round, but we rest nothing on that: the paired gap is +0.062 with 95% CI [−0.018, +0.142] (94% of bootstrap replicates positive), an interval including zero, and its sign does not survive check (iii) below. That interval is the *final* round’s, and the qualification is narrower than it may read: the paired gap excludes zero at every round through round 2 (+0.089, 95% CI [+0.011, +0.167]), and only at the last round does it straddle zero, as the eligible population shrinks and every error falls. We still rest nothing on it, because check (iii) removes the excess at every round at once: what it is evidence of is tier pooling, and a round-by-round pattern of a pooling artifact is still an artifact. Fig. 1 therefore draws the marginal intervals, which overlap throughout, and leaves the paired comparison to this paragraph.

One limit, then three robustness checks. The limit is the pool: these 30 workers are one vendor, two tiers, and three prompt variants, so this measures only the pessimistic half of the claim above (that a resampled same-family worker inherits the same hard points) and cannot test the optimistic half, that genuinely different model families satisfy the condition; that remains the open empirical question.

(i) *Granularity.* At one worker per arm (seed majority within arm, $k=6$) the picture is unchanged, $m^*=6=k$ with 28/55 tasks violating, so the result is not an artifact of counting seeds as workers. (ii) *Threshold.* At SWE-bench’s own bar ($\tau=1.0$, every test passing; §5.1) the statistic is identical, $m^*=30=k$ with 23/55 violating. That is expected rather than reassuring, since raising τ can only turn a solved task into a failed one and m_i is monotone in it, but it means the headline does not rest on the looser threshold.

(iii) *Tier pooling*, which is where the two halves of the result come apart. Restricting to a single tier leaves the overlap at its ceiling on both sides ($m^*=15=k$ over Sonnet’s 15 workers, $m^*=15=k$ over Haiku’s), so the headline is not an artifact of mixing a strong tier with a weak one. The error comparison does not survive: within a tier the vote matches the average worker almost exactly (Sonnet 0.145 against $\bar{\epsilon}=0.147$, 8/55 violating, gap −0.001, CI [−0.039, +0.038]; Haiku 0.545 against 0.565, 30/55 violating, gap −0.019, CI [−0.057, +0.017]). Mixing tiers inflates the majority-failure count while $\bar{\epsilon}$ averages the two, so the pooled gap’s positive sign is a pooling effect rather than a fact about voting. Finally, τ -thresholded quality is a structural proxy for the functional correctness Prop. 6’s ± 1 label denotes (§6.1); Part C cannot substitute, since its sessions share no task instances across workers and m_i is undefined there.

Fig. 1 in §4 plots the distribution and, in its right panel, the per-round errors with a cluster-bootstrap interval on every bar: the vote’s error, $\bar{\epsilon}$, and the Markov bound `cor_overlap_markov_bound`,

whose interval is a deterministic $2\times$ of $\bar{\epsilon}$'s. Those are marginal intervals and they overlap at every round, which is the weaker of the two comparisons available; the paired test is the one reported above, and check (iii) is what attributes the excess to tier pooling.

A separate route to the reweighted edge hypothesis. Prop. 17's hypothesis can also be discharged directly, at a stated cost. If AdaBoost's reweighting concentrates mass by at most a factor κ versus uniform ($D_t(i) \leq \kappa/N$ for all i) and worker t 's error *under the uniform distribution* is at most $(\frac{1}{2} - \gamma)/\kappa$, then $\text{err}_t \leq \frac{1}{2} - \gamma$ as required (`lem_bounded_reweighting_edge`). The concentration cap κ need not be assumed: since $D_t(i) = \exp(-y_i f_t(x_i)) / \sum_j \exp(-y_j f_t(x_j))$ is a softmax of the margins, any bound M on the margins yields $\kappa \leq e^{2M}$ (`D_le_of_margin_bound`), and with ± 1 outputs $M = \sum_{s < t} |\alpha_s|$ always works, giving

$$\kappa \leq \exp\left(2 \sum_{s < t} |\alpha_s|\right)$$

as a function of confidences the algorithm already logs (`cor_rho_from_confidences`), and hence a version of the edge conclusion with no κ hypothesis at all (`cor_reweighting_edge_from_confidences`). What this buys is precise and limited: κ moves from a run-level regularity *assumption* to a *computable* quantity. It does not become small. The bound grows exponentially in the number of rounds, so the uniform-error requirement it implies, $(\frac{1}{2} - \gamma)e^{-2 \sum_{s < t} |\alpha_s|}$, tightens exponentially and is close to vacuous after many rounds. Whether κ is small on a given run is an empirical question this does not answer.

C.3 Reusing a Pool: A Horizon, or an Interpolating Cone

Reuse cannot supply the edge. One structural question about multi-round weighted aggregation admits a sharp negative answer, and the reweighting recursion is what makes it askable: because D_t is defined in closed form, $D_{t+1}(i) = D_t(i)e^{-\alpha_t y_i h_t(x_i)} / Z_t$ is a derived identity rather than a separate definition (`D_succ_eq`).

Proposition 19 (Immediate Reuse Zeroes the Edge). *Run one round with worker h_t at the optimal confidence $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$, where $\epsilon_t = \text{err}_t \in (0, 1)$. Then worker h_t 's weighted error against the distribution its own incorporation produced is exactly $\frac{1}{2}$:*

$$\sum_{i: y_i h_t(x_i) = -1} D_{t+1}(i) = \frac{1}{2}.$$

Its edge on the next round is therefore exactly zero.

Proof sketch. Summing the recursion over the points h_t gets wrong, where the tilt is the constant e^{α_t} , gives $\epsilon_t e^{\alpha_t} / Z_t$ (`reused_worker_err_eq`). At the optimal α_t the two halves of Z_t are equal, $(1 - \epsilon_t)e^{-\alpha_t} = \epsilon_t e^{\alpha_t}$, since $e^{2\alpha_t} = (1 - \epsilon_t)/\epsilon_t$ (`optimal_alpha_balances`); hence $Z_t = 2\epsilon_t e^{\alpha_t}$ and the ratio is $\frac{1}{2}$. Machine-checked as `thm_self_reuse_zero_edge`; `exists_zero_edge_reuse_instance` discharges non-vacuity, since round zero starts uniform (`D_zero_eq_uniform`) and a worker right on one of two points has $\epsilon_0 = \frac{1}{2}$. No square roots are needed: the balance identity does all the work. ■

The naive multi-round scheme is ruled out. This makes precise, and strengthens, a claim the discussion below states informally. Correlated workers fail the edge hypothesis via AdaBoost's own stopping rule; Prop. 19 is the extremal instance of that rule—a worker is maximally correlated with itself, and the reweighting it induces is exactly calibrated to neutralize it. So the obvious way to extend Prop. 17 to many rounds—keep consulting the worker you just consulted—cannot sustain the edge hypothesis for even one further round. Prop. 17 is untouched: its workers $h_0, \dots, h_{k-1}$ are each incorporated once, against the distribution current at that time, so no worker is reused there. And this settles only *self*-reuse; whether some *other* member of a reused pool still has an edge is a different question, settled below (Prop. 20). The alignment with Part B's high failure overlap is suggestive rather than confirmatory: Part B runs sequential self-refinement, not AdaBoost reweighting, so the result explains why reuse is structurally unpromising without predicting that measurement.

The cross-worker case: a capacity ceiling, not a correlation obstruction. Prop. 19 leaves open whether some *other* member of a reused pool retains an edge. Two facts settle the shape of the answer. First, there is no per-round obstruction: against any round’s distribution—including the one a reuse produces—a binary worker with error at most $1/N$ exists, hence edge at least $\frac{1}{2} - 1/N$ (`exists_edge_worker_vs_any_round`), so Prop. 19 does not generalize into a one-round impossibility. Second, what does bind is the pool’s *capacity*: collecting the confidences a schedule assigns to each pool member shows the ensemble margin never leaves the pool’s nonnegative cone, a set fixed by the pool and independent of both the number of rounds and the schedule (`lem_pool_span_collapse`). The next two propositions are the two halves of Prop. 7, stated here with the exact hypotheses the main-text summary compresses: this one covers $\epsilon_0 > 0$, and Prop. 21 below covers $\epsilon_0 = 0$.

Proposition 20 (Reused-Pool Horizon). *Run Algorithm 1 with a fixed pool of k workers under any schedule, at the optimal confidences, every scheduled worker having edge $\gamma > 0$ against the distribution current when it is used. If no element of the pool’s nonnegative cone achieves zero-one training error below $\epsilon_0 > 0$, then*

$$\epsilon_0 \leq e^{-2\gamma^2 T}, \quad \text{hence} \quad T \leq \frac{\log(1/\epsilon_0)}{2\gamma^2}.$$

The bound is uniform over the schedule and the confidences, and depends on k only through ϵ_0 . Consequently the edge hypothesis cannot hold at every round: a pool whose cone cannot fit the training data provably fails it by a finite, computable round.

Proof sketch. Grouping the rounds by which pool member they use rewrites the margin $\sum_{t \leq T} \alpha_t h_{\sigma(t)}$ as $\sum_j A_j H_j$, with A_j the total confidence sent to member j (`lem_pool_span_collapse`); each $A_j \geq 0$ because an edge at the optimal confidence forces $\alpha_t \geq 0$ (`alpha_opt_nonneg`), which is AdaBoost’s stopping rule read forwards. So the run’s zero-one loss is the loss of a cone element (`zero_one_loss_eq_spanLoss`), which ϵ_0 bounds below, while Thm. 10 and Cor. 11 bound it above by $e^{-2\gamma^2 T}$. Since a fixed pool supplies one ϵ_0 for every T , a large enough T contradicts the horizon (`cor_no_perpetual_pool_edge`). Machine-checked as `thm_pool_reuse_horizon`; `exists_positive_span_floor` and `exists_pool_reuse_horizon_instance` discharge non-vacuity, the latter exhibiting a pool where a positive cone floor and a surviving edge round hold *jointly*, so the statement is not vacuously true. ■

The obstruction is capacity, not correlation. The mechanism is not that reused workers correlate. It is that a fixed pool has a fixed reachable class, which is Prop. 3’s stationarity dichotomy and its frozen-weight ceiling (Cor. 4) transplanted into the weighted-vote setting. The qualitative version of this is classical. AdaBoost’s minimax duality already gives that a pool is γ -weak-learnable exactly when its convex hull separates every training point with margin at least 2γ [Schapire, 2013]. What is contributed is the *effective* negative half, machine-checked, with an explicit horizon uniform in schedule and confidences; the duality’s converse direction, weak learnability $\Rightarrow$ interpolation with margin, is not formalized. Next, this result itself says nothing when $\epsilon_0 = 0$: a pool whose cone interpolates the training set is untouched by the horizon argument, and such a pool *does* sustain the edge, which is now machine-checked separately, and without duality, as Prop. 21. The two propositions partition the cases by the sign of ϵ_0 , so between them the negative and positive content are both accounted for. Finally, the proof itself is not reuse-specific, since any sequence of workers factors as a pool of size T used once each. What does the work is the *fixity* of ϵ_0 across T , not the repetition of workers; with fresh workers each round no single ϵ_0 survives.

The interpolating case, and why it is the easy half. Prop. 20 is silent when $\epsilon_0 = 0$. That gap closes in the positive direction: a pool whose nonnegative cone fits the training data sustains the edge hypothesis against *every* round’s distribution, at an edge fixed once and for all by the fitting combination’s margin.

Proposition 21 (Interpolating Pools Sustain the Edge). *Let $H_1, \dots, H_k$ be binary workers, and suppose some nonnegative combination A interpolates the sample: $y_i \sum_j A_j H_j(x_i) > 0$ for every i , equivalently $\text{spanLoss}(A) = 0$. Put $\theta = \min_i y_i \sum_j A_j H_j(x_i) > 0$ and $S = \sum_j A_j > 0$. Then*

for every distribution D on the sample there is a pool member j with

$$\sum_{i: y_i H_j(x_i) = -1} D(i) \leq \frac{1}{2} - \gamma, \quad \gamma = \frac{\theta}{2S} > 0.$$

The order of quantifiers is the content: γ is fixed before D is chosen, so the edge holds at every round of any run, whatever the schedule and confidences did beforehand.

Proof sketch. Zero cone loss means a *strict* margin at every point, and finitely many positive numbers have a positive minimum θ (`lem_margin_floor_of_spanLoss_zero`); $S > 0$ because vanishing coefficients would make every margin zero (`lem_span_coeff_sum_pos`). Averaging the pointwise bound $\theta \leq y_i \sum_j A_j H_j(x_i)$ against D and exchanging the two sums gives $\theta \leq \sum_j A_j e_j$, where $e_j = \sum_i D(i) y_i H_j(x_i)$ is worker j 's D -weighted margin. A nonnegative combination cannot exceed its coefficient sum times its largest term, so $\theta \leq S \max_j e_j$ and some $e_j \geq \theta/S$. For a binary worker $e_j = 1 - 2\epsilon_j$ (`lem_weighted_margin_eq`), whence $\epsilon_j \leq \frac{1}{2} - \theta/2S$. ■

The positive half, proved without duality. No duality is used. AdaBoost's minimax duality is needed for the *converse* (weak learnability $\Rightarrow$ interpolation with margin), which remains unformalized; the direction proved here is averaging plus a pigeonhole.

First, Prop. 21 and Prop. 20 are complementary halves of a dichotomy on ϵ_0 rather than overlapping claims: the horizon's floor hypothesis together with $\epsilon_0 > 0$ is outright incompatible with the existence of an interpolating cone element (`cor_pool_horizon_floor_excludes_interpolation`), so the restriction to $\epsilon_0 > 0$ is the boundary between the two cases, not an artifact of the proof. Second, and more consequential, this is a *per-round* statement: at every round some member has the edge. It does not exhibit a schedule and confidence sequence realizing a full run, which would require defining the two by simultaneous recursion (α_t is pinned by the optimal-confidence rule to a quantity that reads D_t , which in turn reads the schedule and confidences before t) and would additionally need to exclude a *perfect* pool member, since Cor. 11 assumes $\epsilon_t > 0$ and an interpolating pool is precisely the regime where a member may misclassify nothing. So the multi-round weighted-vote run is still not formalized; what closes is the $\epsilon_0 = 0$ case.

C.4 Are the Two Conditions Equivalent?

Are the two conditions equivalent? Prop. 17's reweighted edge hypothesis and Prop. 6's overlap condition $m^* < k/2$ have stood alongside each other with no relation established between them. They are in fact independent.

Proposition 22 (The Two Conditions Are Independent). *Neither the reweighted edge hypothesis nor the bounded-overlap condition implies the other. Both directions are witnessed on four points with three workers. (i) Three workers with reweighted errors $\frac{1}{4}, \frac{1}{6}, \frac{2}{5}$ —each at most $\frac{1}{2} - \frac{1}{10}$, so the edge hypothesis holds with $\gamma = \frac{1}{10}$ at every round—in which one point is failed by two of the three, so $2m^* = 4 \geq k$ and the overlap condition fails. (ii) Three workers with $m^* = 1 < k/2$, hence a correct unweighted vote, in which one worker has error $\frac{3}{4}$, so no $\gamma > 0$ makes the edge hypothesis true.*

Proof sketch. In (i) the errors are the genuinely reweighted ones, not uniform-distribution stand-ins. At the optimal confidence the reweighting recursion loses its exponentials—a failing point's weight is divided by $2\epsilon_t$ and a passing point's by $2(1 - \epsilon_t)$ (`D_succ_fail`, `D_succ_pass`, both from `D_succ_eq` and `optimal_alpha_balances`)—so rational errors keep the entire distribution rational and the three rounds are settled by arithmetic. With failure sets $\{0\}, \{1\}, \{0, 2\}$ the successive distributions are $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$, $(\frac{1}{2}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6})$, $(\frac{3}{10}, \frac{1}{2}, \frac{1}{10}, \frac{1}{10})$, and point 0 is failed by workers 0 and 2 (`thm_edge_not_imply_overlap`). In (ii), one worker failing three of the four points has error $\frac{3}{4}$ against the uniform round-zero distribution while no point is failed by more than one worker (`thm_overlap_not_imply_edge`); note $\frac{3}{4}$ lies strictly inside $(0, 1)$, so what fails is the edge bound itself rather than a degeneracy at $\epsilon \in \{0, 1\}$. Packaged as `thm_edge_overlap_independent`. ■

C.5 The Probabilistic (Condorcet) Route

The probabilistic route. Diversity also has a probabilistic formulation—the Condorcet/Hoeffding bound, where independent worker errors yield exponential decay for the majority vote. It can be had without leaving Prop. 6’s combinatorial setting.

Proposition 23 (Condorcet Bound for Independent Errors). *Let $2m$ workers fail independently, worker i with probability $q_i \leq \frac{1}{2} - \gamma$ for some $\gamma \in (0, \frac{1}{2})$. Then the unweighted majority vote errs with probability at most*

$$(1 - 4\gamma^2)^m \leq e^{-2\gamma^2 \cdot 2m}.$$

Proof sketch. Write the weight of a failure pattern $\mathcal{F}$ as the product $\prod_{i \in \mathcal{F}} q_i \prod_{i \notin \mathcal{F}} (1 - q_i)$ (`wt`), and the vote’s error as the total weight of the patterns with $2|\mathcal{F}| \geq 2m$ (`voteErr`). The algebraic identity $\prod_i (f_i + g_i) = \sum_{\mathcal{F}} \prod_{i \in \mathcal{F}} f_i \prod_{i \notin \mathcal{F}} g_i$ does all the work: at $f_i = q_i$, $g_i = 1 - q_i$ it shows the weights sum to 1 (`lem_wt_sum_one`), and at $f_i = q_i r$ it factorizes the tilted sum (`lem_wt_tilt_sum`). Taking $r = (1 - \gamma')/\gamma'$ with $\gamma' = \frac{1}{2} - \gamma$, every counted pattern has $r^{|\mathcal{F}|} \geq r^m$, so the error is at most $r^{-m} \prod_i (q_i r + 1 - q_i) \leq r^{-m} (2(1 - \gamma'))^{2m} = (4\gamma'(1 - \gamma'))^m = (1 - 4\gamma^2)^m$, and $1 - 4\gamma^2 \leq e^{-4\gamma^2}$ finishes. Machine-checked as `thm_condorcet_majority_bound`, with `exists_condorcet_instance` for non-vacuity. No moment-generating function is evaluated and no logarithm appears: r is written down directly. ■

The price of assuming independence. The constant $e^{-2\gamma^2 k}$ is exactly Cor. 11’s, so Prop. 23 composes with Prop. 17 with no further analysis—the same k_0 works. Independence was initially *encoded* in the product form of the pattern weight rather than derived. It is now derived. Prop. 24 places the workers on a genuine product of Bernoulli measures over $\{0, 1\}^{2m}$, obtains the independence of the per-worker failure indicators from Mathlib’s product-measure machinery, and identifies the measure of the majority-failure event with the very quantity Prop. 23 bounds. So the stronger reading of “independent” is earned, and by an identification rather than by re-deriving the bound from Hoeffding’s inequality: the combinatorial proof was always a statement about this measure, and that is now a theorem instead of an assertion. What no proof can discharge is the decision to model worker failures by a product measure at all—that is the Condorcet jury setup itself. Nor is the assumption innocent empirically: independent failures across workers is precisely what Part B’s 23/55 (42%) majority-failure rate fails to exhibit (§5.1), and what Prop. 6’s overlap condition constrains without assuming.

Proposition 24 (Condorcet Bound with Independence Derived). *Let $\mu = \bigotimes_{i=1}^{2m} \text{Bernoulli}(q_i)$ on $\{0, 1\}^{2m}$, coordinate i recording whether worker i fails. The per-worker failure indicators are independent under μ , and for $q_i \leq \frac{1}{2} - \gamma$ with $\gamma \in (0, \frac{1}{2})$,*

$$\mu(\{\omega : \#\{i : \omega_i = 1\} \geq m\}) \leq e^{-2\gamma^2(2m)}.$$

The event’s measure equals the `voteErr` of Prop. 23, so the two propositions bound the same quantity.

Proof sketch. Independence is not assumed: it is Mathlib’s `iIndepFun_pi` applied to the coordinate projections of a product measure (`lem_coord_iIndepFun`). The measure of a single failure pattern is the product a product measure actually has, which is exactly the pattern weight `wt` (`lem_pattern_measure`, via `Measure.pi_singleton`); summing over the patterns in which at least half the workers fail, reindexed along the bijection between patterns and failure sets, identifies the event’s measure with `voteErr` (`lem_failure_event_eq_voteErr`). Prop. 23 then transports verbatim. The bound is not re-proved, and Hoeffding’s inequality is not used: the sample space is a finite product of finite spaces, so no centering step or measurability side condition arises. ■

C.6 Status: What Is Settled, and What Is Not

What remains open. The comparison above is to the *weak-learning threshold* $\frac{1}{2} - \gamma$, not to whichever single worker happens to be best in a given instantiation: a worker that is accidentally already excellent is not beaten by this result, nor need it be. Best-of- k hard selection (Prop. 5) already reaches that worker’s ceiling exactly. The regime where weighted aggregation earns its keep is the

reverse one: many individually-mediocre workers, none dominating the others, where hard selection is stuck at the least-bad of a bad set while weighted voting escapes that floor, given the diversity condition that each worker keeps beating the *reweighted* distribution which up-weights points previous workers got wrong. Correlated or identical workers fail this hypothesis via AdaBoost’s own stopping rule ($\epsilon_t \geq \frac{1}{2} \Rightarrow \alpha_t \leq 0$, Algorithm 1), not via an assumption bolted on separately. Diversity is encoded in the edge hypothesis itself, not assumed on top of it.

Three questions about multi-round weighted voting are settled here, and not all of what settles them is ours. *Reuse* is bounded: consulting the same worker twice zeroes its edge outright (Prop. 19), and a reused *pool* either interpolates the sample (in which case some member has an edge against every distribution, Prop. 21) or does not, in which case the edge hypothesis fails by round $\log(1/\epsilon_0)/2\gamma^2$ (Prop. 20). That dichotomy is Prop. 7 in the main text; its qualitative half follows from AdaBoost’s minimax duality and is not ours, while the effective form with an explicit horizon is. *Diversity* has a probabilistic route as well as a combinatorial one: the Condorcet/Hoeffding majority bound holds at the same constant $e^{-2\gamma^2 k}$ as Cor. 11 (Prop. 23), with the independence of the per-worker failure indicators derived on a product of Bernoulli measures rather than encoded in the pattern weight (Prop. 24). *The two conditions* are logically independent: neither $m^* < k/2$ nor the reweighted edge hypothesis implies the other (Prop. 22), and the concentration cap κ the latter needs is derivable from logged confidences rather than assumed.

What remains outside the formalization is not a gap in a proof. The multi-round weighted-vote *run*—a schedule and confidence sequence exhibited outright, as distinct from the per-round edge feeding it—is not formalized, and additionally requires excluding a perfect pool member since Cor. 11 assumes $\epsilon_t > 0$. And choosing a product measure over worker failures *is* the Condorcet jury setup: it is a modelling decision no formalization can discharge, and precisely the assumption Part B’s 23/55 (42%) majority-failure rate shows real workers violate.

D Part A: Additional Figures

Part A’s scope. Labels throughout Part A are synthetic—a fixed linear separator applied to real SWE-bench TF-IDF features plus noise—so no claim about code, correctness, or LLM behavior is being tested here; the same curves would result from running the identical script on any other feature distribution. Part A’s role is an implementation sanity check, confirming the training-error, margin, and generalization machinery is coded correctly before that machinery is applied to Part B/C data, and it supplies the positive-edge reference trajectory against which Part B’s measured decay in $\tilde{\gamma}_t$ (§5.1) becomes interpretable as a property of real agents rather than of a broken measurement pipeline. Note the estimand shift: Part A’s stumps have a genuine solve-edge $\gamma_t > 0$, whereas Part B’s $\tilde{\gamma}_t$ scores improvement over the previous draft and goes negative largely because the loop stops changing the artifact—so the two trajectories are comparable in shape, not in sign. The claim that boosting mechanics govern real code generation is carried entirely by Parts B and C, not by Part A.

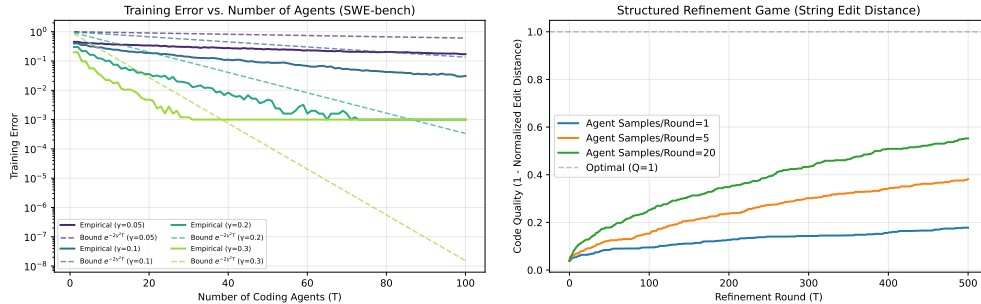


Figure 2: **Left:** training error vs. number of weak coding agents for four target edges γ ; solid = empirical (mean of 5 trials), dashed = bound $e^{-2\gamma^2 T}$, confirming Cor. 11. **Right:** code quality under the refinement game (Levenshtein-distance optimization), confirming Thm. 2; quality increases monotonically with diminishing returns, remaining bounded below 1.0, with more capable agents converging higher.

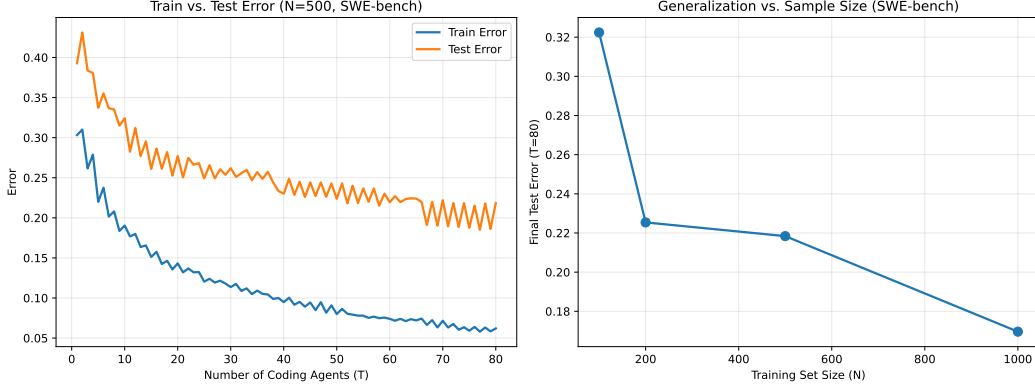


Figure 3: Left: Train vs. test error for $N = 500$ over 80 boosting rounds. Test error decreases then plateaus, consistent with the generalization bound. Right: Final test error decreases monotonically with training set size (averaged over 5 seeds on the full SWE-bench split with a disjoint held-out test set), consistent with the $O(\sqrt{d/N})$ term.

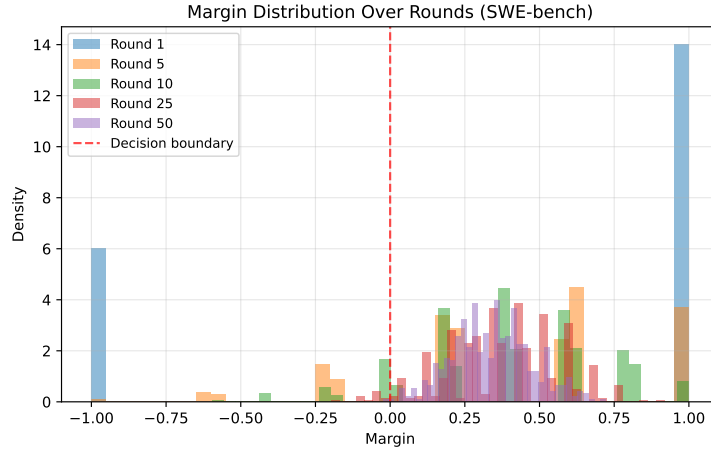


Figure 4: Margin distribution over boosting rounds (edge-calibrated weak agents). Round 1 is a single learner with saturated ± 1 margins (roughly half misclassified); over successive rounds the negative-margin mass collapses and the distribution concentrates on the positive side of the boundary. This concentration of margin mass away from the decision boundary is the mechanism by which boosting achieves good generalization despite adding more weak learners.

E Part B: Harness Verification, Task Pool, and Power Analysis

Harness verification and task pool (methods disclosure). Our evaluator is not SWE-bench’s own. A T -round refinement experiment needs per-round scoring inside a container that stays alive between rounds; the official harness is a *batch* evaluator—`run_evaluation.py` grades one patch per instance from a predictions file—so re-invoking it per round per task across $\approx 6,600$ evaluations was not viable, and we built the loop ourselves, scoring `--junit-xml` reports directly. We stand by that decision and not by its execution: SWE-bench also ships per-repository test commands (`MAP_REPO_VERSION_TO_SPECS`) and per-repository log parsers (`MAP_REPO_TO_PARSER`), both importable independently of its CLI and its container lifecycle, and we reimplemented that knowledge instead of importing it.

A gold-patch audit—apply the *known-correct* SWE-bench patch and check it scores 1.0—of the originally-sampled $K=50$ pool found 23/50 (46%) tasks failing deterministically, none of them model failures, separable into 5 root causes. Three are ours outright. `pytest`

is not the test runner for Django or SymPy at all (9 tasks): their official commands are `./tests/runtests.py --verbosity 2` and `bin/test -C --verbose`. An older pinned `pytest` rejects the `--no-header` flag we passed universally (3), which the official specs apply only to `seaborn`, where the pinned version accepts it. And ANSI color codes broke a text-based pass/fail line parser (3), which `astropy`’s official command forestalls by setting `console_output_style=classic`. One is partly genuine and independently known: `parametrize-ID` drift between `pytest`’s auto-generated collection ids and SWE-bench’s recorded ids (7). One is mundane: a network-dependent test that hangs inside a sandboxed evaluator with no internet access (1). We record the split because the count is a fact about *our* evaluator rather than about the benchmark: read as a benchmark defect rate it would be simply wrong, and the distinction is the whole reason the breakdown is here rather than a bare failure count.

We fixed the two shared-root-cause bugs (replacing text-parsing of colorized `pytest -v` output with `--junit-xml` scoring, and fuzzy-matching `parametrize` ids via a `pytest --collect-only` pass) and verified the fix against the gold patch on every previously-flagged task (all now score `fraction_passing=1.0`); those two are the transferable part of the exercise. We then excluded Django and SymPy entirely, not because their environments are defective but because a `pytest`-only evaluator cannot run them, plus the one network-dependent task, and backfilled 15 gold-patch-verified replacement tasks to reach $K=55$; every active task’s Docker image is now digest-pinned. All Part B results use this harness-fixed, gold-verified $K=55$ pool, which is therefore `pytest`-native by construction. That is a property of our tooling that shaped the pool, and the reason a follow-up should add per-repository dispatch and restore both repositories rather than inherit this selection.

One divergence survives inside the retained pool. SWE-bench runs `sphinx` tests through `tox --current-env -epy39 -v --` rather than bare `pytest`, and `sphinx-doc` is one of the 10 retained repositories. What bounds the risk is the audit itself: every retained task, `sphinx` included, scores `fraction_passing=1.0` under the gold patch, so bare `pytest` is collecting and running those tests rather than silently reporting nothing. Agreement with the official grader on the retained pool is established only through that gold-patch check, not by running both harnesses side by side.

Table 2: Part B: 2×3 repeated-measures ANOVA (model $\times$ feedback mode) on final-round quality, $K=55$ tasks $\times$ 5 seeds. Summarized in §5.1. The model effect’s Cohen’s f carries 95% CI [0.738, 1.319], consistent with the near-complete tier separation in solve rates rather than with a computation artifact.

Effect	F	df	p	partial η^2	Cohen’s f
Model (Sonnet vs. Haiku)	53.46	(1, 54)	0.0001	0.497	0.99 (huge)
Feedback mode	0.50	(2, 108)	0.61	0.009	0.10 (small)
Model $\times$ mode	0.81	(2, 108)	0.45	0.015	0.12 (small)

Per-arm solve rates. The ranges quoted in §5.1 resolve as follows at $\geq \tau=0.6$ with 95% Wilson intervals. Haiku: independent 24/55 (44%) [31, 57], blind 26/55 (47%) [35, 60], diagnostic 27/55 (49%) [36, 62]. Sonnet: independent 47/55 (85%) [74, 92], blind 47/55 (85%) [74, 92], diagnostic 49/55 (89%) [78, 95]. The intervals within a tier overlap heavily, which is the same fact the mode term’s null records. At SWE-bench’s own criterion ($\tau=1.0$) the weak tier’s mode ordering inverts, which is why §5.1 reads the mode contrast as noise: Haiku independent 21/55 (38%) against diagnostic 16/55 (29%), while Sonnet’s diagnostic arm gives 41/55 (75%).

The dependent variable, and an asymmetry in it. The dependent variable is final-round quality averaged over seeds. It is therefore threshold-free, which is why τ moves the solve-rate tallies of §5.1 and nothing else: the ANOVA’s DV is continuous quality, and $\tilde{\gamma}_t$ compares $q_{i,t}$ against $q_{i,t-1}$ directly rather than against any threshold. Because the harness back-fills the rounds after a task first passes (Appendix E.1), a solved row’s final round is its running best, whereas an unsolved row contributes whatever its last attempt scored—which may sit below its own earlier best, on 5 and 12 rows. The stronger tier solves more rows, so the asymmetry runs in the same direction as the tier effect and cannot be dismissed on inspection. Re-running the identical ANOVA with every row contributing its running best moves the tier effect from partial $\eta^2=0.497$ to 0.514, and leaves mode and interaction non-significant: the effect does not rest on the asymmetry.

Task pool. The active $K=55$ pool spans 10 SWE-bench Lite repositories (astropy, flask, matplotlib, pylint, pytest, requests, scikit-learn, seaborn, sphinx, xarray); Django and SymPy are excluded entirely and one network-dependent task is excluded. Of the originally-sampled 50 tasks, 40 are retained unchanged and 15 are gold-patch-verified replacements backfilled to reach $K=55$. Exhaustive per-task, per-arm, per-seed quality trajectories for all $6 \text{ arms} \times 55 \text{ tasks} \times 5 \text{ seeds}$, together with the committed analysis script that reproduces every Part B table and figure directly from that data, are in the supplementary repository.

Power analysis (full detail). *This null is underpowered, not evidence of zero effect.*

A case-resampling bootstrap (30,000 reps/ K , preserving each task’s full 6-arm profile) at the observed effect size finds only 11% power (mode) and 17% power (interaction) at the $K=55$ we ran; reaching 80% power requires $K \approx 467$ (mode) and $K \approx 364$ (interaction), roughly $7\text{--}9\times$ the current sample. An independent, quantified reason compounds the power problem regardless of K : seed-to-seed noise within a fixed (task, model, mode) cell averages $SD \approx 0.096$, about $1.7\times$ the mean between-mode signal (mean $|\text{diagnostic} - \text{blind}|$ per task ≈ 0.057). The sampling-noise floor exceeds the effect being measured.

This noise is a deliberate, explained design choice, not an oversight: generation temperature 0.25 is applied uniformly across all three arms, but it is *structurally required* only for independent, whose prompt is identical every round (no prior patch, no critique); at temperature 0 it would degenerate into T identical greedy draws. `blind` and `diagnostic` have no such requirement, since their prompts already change round to round with the carried-forward patch and critique; a single shared temperature was chosen for simplicity over doubling the design into temperature-decoupled arms. A future rerun could decouple this, running temperature 0 for `blind/diagnostic` and temperature >0 only for independent, as a variance-reduction change to the design, not a reanalysis of the data collected here.

Why report a null this underpowered at all? First, the required- K and noise-to-signal numbers above are themselves informative: they tell a future replication exactly how large a sample the effect needs, which a silently-dropped ablation would not. Second, the capability effect (Table 2) is properly powered at this same K , so the ablation is not a uniformly underpowered design—only the mode and interaction terms are, and the significant capability result should not be read as implying the whole design was adequately powered. Third, the null is the empirical basis for the “spend on the first attempt” guidance (§6): reporting it with its power caveat attached is more useful to a practitioner than either suppressing it or overclaiming it as a confirmed non-effect.

The point-estimate crossover (not significant; a hypothesis, not a finding). Haiku’s mean quality increases monotonically with more carried-forward context (independent $0.432 < \text{blind } 0.449 < \text{diagnostic } 0.466$), while Sonnet’s `blind` arm is its *worst* of the three (independent 0.875 , `blind` 0.856 , `diagnostic` 0.871)—a crossover, not merely noise around a flat line (Fig. 5). Every formal test of this ordering is non-significant (linear contrast $F = 0.56\text{--}2.16$, $p = 0.15\text{--}0.46$ across gating choices; $\text{model} \times \text{linear}$ interaction $p = 0.13\text{--}0.33$), so we do not claim it, and we explicitly do *not* attribute Sonnet’s inversion to “solved early, then regressed later”—we tested that mechanism directly against the per-round trajectories and it runs the wrong direction to explain the gap (Sonnet’s `blind` gives back *less* to late-round regression than `independent` does, 0.0091 vs. 0.0105).

E.1 Decomposing the Measured Edge

The main text reports the per-round edge on measured cells and the pooled series alongside it. This is the detail behind both, and the reason the pooled number cannot stand alone: γ ’s *sign* is the quantity the weak-learning condition turns on, and in this setting the sign is determined largely by something other than what the condition is about.

Rounds the experiment did not run. The harness stops calling the model on the round a task first passes, then back-fills the remaining rounds with a hardcoded $q=1$ so the T -wide matrix stays rectangular. Those cells record no patch, no evaluation and no model call. They are 27.6% of Haiku’s matrix and 53.9% of Sonnet’s, and because γ scores a tie at $-\frac{1}{2}$, each one enters as a failure to improve. They account for 33–45% (Haiku) and 74–86% (Sonnet) of each round’s tie mass. The pooled estimator is therefore driven downward in proportion to the solve rate, which inverts

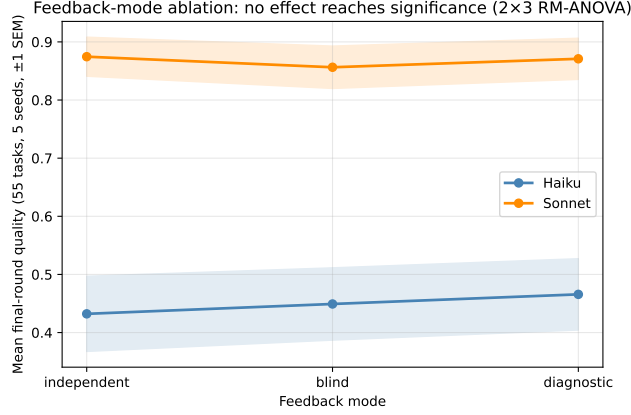


Figure 5: Part B feedback-mode ablation: mean final-round quality by mode (independent $\rightarrow$ blind $\rightarrow$ diagnostic), $K=55$ tasks $\times$ 5 seeds, ± 1 SEM. Haiku increases monotonically; Sonnet’s blind arm is its worst of the three—a non-significant crossover (see text), not a confirmed finding.

the comparison it looks like it supports: Sonnet solves far more tasks, back-fills far more cells, and posts a *more* negative pooled edge at round 3 than Haiku ((0, -0.33, -0.41, -0.46) against (0, -0.39, -0.46, -0.45), CIs $[-0.373, -0.285]$, $[-0.449, -0.365]$, $[-0.482, -0.435]$ and $[-0.449, -0.322]$, $[-0.485, -0.431]$, $[-0.478, -0.416]$) while being the stronger refiner on every cell where a call was made. That pooled sign is therefore not reportable as a headline (§5.1): it is an artifact of the harness rather than a property of the loop.

Ties versus regressions. The indicator uses a strict $>$, so “not improved” pools two unlike outcomes: quality identical to the previous round, and quality actually lower. Splitting them on the diagnostic arm gives per-round tie rates of (0.887, 0.945, 0.949) for Haiku and (0.800, 0.898, 0.953) for Sonnet, against regression rates of (0.004, 0.015, 0.000) and (0.029, 0.011, 0.007). Net of the back-filling above, the genuine no-op rate is 52–59% (Haiku) and 13–21% (Sonnet), still the dominant outcome and the figure that actually supports “refinement turns idempotent”. Whole trajectories tell the same story: 80.7% of Haiku’s 275 (task, seed) rows and 69.1% of Sonnet’s never change quality at any round, and only 5 and 12 rows respectively dip even once. A refiner that has fully converged—every round identical, never a regression—attains $\gamma = -\frac{1}{2}$ exactly, the floor of the estimator’s range. So $\gamma < 0$ here is evidence that the loop has stopped changing the artifact, which is what Prop. 3(i) predicts; it is *not* evidence that patches are anti-correlated with correctness, which is what $\epsilon_t > \frac{1}{2}$ means in AdaBoost and what “the weak-learning condition fails” would ordinarily convey. Both readings are consistent with the same number, and only the first is supported.

The edge among tasks that can still move. A row already at $q=1$ cannot improve, so it enters γ ’s denominator as a row that did not improve when it is really a row that could not. Conditioning each round on $q_{i,t-1} < 1$ removes that: the eligible population falls 194 $\rightarrow$ 158 of 275 (Haiku) and 113 $\rightarrow$ 49 of 275 (Sonnet) as rows saturate, and the conditional edge is (-0.345, -0.435, -0.411) with CIs $[-0.426, -0.247]$, $[-0.474, -0.383]$, $[-0.460, -0.348]$ for Haiku against (-0.084, -0.143, -0.276) with CIs $[-0.173, +0.021]$, $[-0.286, +0.028]$, $[-0.389, -0.125]$ for Sonnet (same 9,999-resample task-cluster bootstrap and the same seed as the pooled intervals, so the two are computed over identical cluster draws; numerator and denominator are resampled together, so the denominator’s own sampling variation is carried rather than held fixed).

Haiku’s conditional intervals all sit strictly below zero, so its failure to improve is not an artifact of saturation. Sonnet’s first two *include* zero, so on tasks with room left we cannot reject a zero-or-slightly-positive edge in the early rounds, and only by round 3 is the negative sign unambiguous. That is a genuine qualification of the main claim. And the tier separation therefore survives conditioning, which is the more direct measurement of “capability, not loop depth” than the pooled series provides—a stronger model spends its rounds running out of headroom, a weaker one spends them failing to exploit the headroom it has.

A correction, not a clean unconditional edge. It is the estimator over cells where a model call happened, so relative to the pooled series it is a correction and not merely a decomposition: excluding a back-filled round is declining to average over a non-event, not conditioning on an outcome. The paragraph above is why: counting the $q=1$ rows as rows the model failed to improve reads a non-event as a negative outcome, when the model was never asked. What it is still *not* is a clean unconditional edge. Excluding those cells also excludes the rounds that would have followed a pass, and those rounds were never run, so no reweighting recovers them; the remaining population is additionally selected against rows whose earlier rounds went well, which is a real difference in population rather than a nuisance to adjust away.

The confidence-rated reading. A round that leaves quality unchanged is closer to an *abstention* than to an error, and Schapire and Singer [1999] give the standard treatment: with per-round weights ω_+ , ω_- and ω_0 over measured cells, the optimally-weighted round contributes $Z_t = \omega_0 + 2\sqrt{\omega_+\omega_-}$ to the training-error product, and $Z_t < 1$ whenever improvements outnumber regressions. On this arm the product over rounds 1–3 is 0.81 (Haiku) and 0.67 (Sonnet): the bound descends, slowly, which is what saturation looks like in boosting’s own terms. The binary form $\exp(-2\sum_s \max(0, \tilde{\gamma}_s)^2)$ pins at 1 instead, and that flatness is a property of a formulation with no representation for abstention rather than of the loop. Conditional on a round changing anything at all, the edge is strongly positive throughout (+0.233 to +0.500 for Haiku, +0.346 to +0.393 for Sonnet), reported for completeness only, since that denominator is both small and selected on the outcome being measured. The denominator shrinks with t (to 49 rows for Sonnet at round 3), so the late-round conditional intervals are correspondingly wide, and we read the round-3 point estimates as the least reliable of the six. Nothing here changes what the criterion requires: Thm. 2 and Prop. 3 need only a per-round improvement property, and $\bar{\eta}_t$ (§5.1) remains positive and shrinking under both tiers either way.

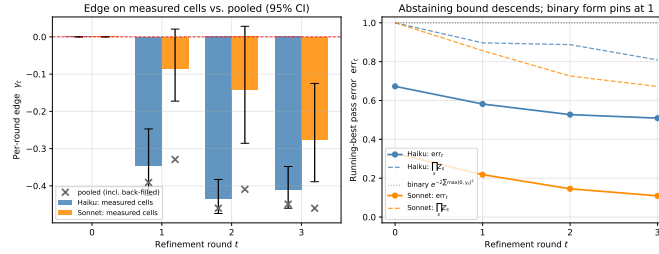


Figure 6: Part B, per-round edge on the diagnostic arm; cited from §5.1. **Left:** bars are $\tilde{\gamma}_t$ over cells where a model call happened, with 95% CIs (9,999-resample cluster bootstrap over the 55 tasks); grey crosses are the pooled $\tilde{\gamma}_t$ over all cells, the gap being the back-filling described above. Round 0 has no predecessor, so no bar; Sonnet’s first two intervals cross zero. **Right:** running-best error (solid) against the abstaining bound $\prod_s Z_s$ (dashed), which descends because an unchanged round is charged $Z_s < 1$ rather than as an error; the binary form $\exp(-2\sum_s \max(0, \tilde{\gamma}_s)^2)$ (dotted) pins at 1.0, an artifact of having no representation for abstention.

F Part C: Additional Findings and Figures

Data availability. Four anonymized session/commit datasets underlie Part C, all in the supplementary repository and all written by the same sanitization pipeline (`scripts/anonymize_sessions.py`): `data/company_sessions.json` for the AI-tier sessions, `data/company_control_sessions.json` for the pre-AI human baseline of Finding 3, and `data/company_within_era_ai_sessions.json` with `data/company_within_era_nonai_sessions.json` for the two sides of the within-era contrast below. Real repository names, commit hashes, and author identities are replaced with opaque synthetic labels before release; timestamps are truncated to day granularity, developer labels are salted hashes, commit SHAs are counter-substituted, and only structural metrics (diff sizes, commit counts, model trailer, coarse dates) survive (full sanitization audit in the repository README).

The two label kinds are not symmetric, and the difference is easy to miss because the labels look alike. *Developer* labels are HMAC(salt, identity), so under one salt and one roster the same person

carries the same `dev_` label in every file, which is what makes the paired within-developer analyses below possible at all. *Repository* labels are *positional*: they are lettered per extraction run by HMAC rank over that run’s repository list, so `repo_A` denotes a *different* repository in each file, and the label sets overlap without the repositories doing so. Merging datasets on `repo` would therefore fuse unrelated repositories into one random-effect level; `scripts/part_c_control_stats.py` namespaces the labels per file (`sim.namespace_repos`) before any pooled model is fit, and third-party reanalysis must do the same.

Every Part C number quoted in this paper is recomputed from those four files by `scripts/part_c_control_stats.py` into `data/part_c_control_stats.json`, which is the sole source for the macros this text expands; a companion `--check` gate fails the build if any quoted value drifts from the recomputed one.

Why the linkage compares instants, not strings. The commit-to-session linkage compares parsed UTC instants, and the distinction is not cosmetic on this cohort: contributors span 11 distinct UTC offsets, so under a comparison of ISO-8601 timestamp *strings* a commit written at `...T10:00:00+05:30` sorts after one written at `...T09:00:00-04:00` while in fact preceding it by four hours. Commits then fall outside their own session’s window and go unlinked—124 control and 15 AI-tier commits, dropped from the per-session churn aggregates without any error being raised. Comparing instants takes unlinked commits to zero on both cohorts, and every Part C figure is computed on that linkage.

Cohort composition, and the spread inside each tier label. Finding 2 contrasts Claude Opus (stronger) against Claude Sonnet (weaker)—a different pair from Part B’s, where Sonnet is the *stronger* of the two, so the two parts’ “capability contrast” is not the same comparison. That pair is also not the whole cohort: 14 sessions fall outside it, 9 Haiku and 5 with no resolvable trailer. And each label names a model *family* rather than a checkpoint—the trailers record five Opus and three Sonnet versions spanning generations 4.5 through 5 in both families, which the two-bin contrast pools—so within-family capability spread is uncontrolled, which is what the version-resolved re-test below addresses. The gap is also robust to the choice of test, giving Mann–Whitney $p=0.91$ alongside the permutation p quoted in §5.2.

Finding 4—Geometric churn-decay model captures the coarse shape (consistency check). The geometric churn-decay (log-normal) model, fitted to consecutive commit pairs, yields $\rho=0.77$ (95% bootstrap CI: [0.65, 0.92]) and $\sigma=2.39$ (95% CI: [2.24, 2.54]). This CI excludes 1.0, giving some support that the decay rate is genuinely below unity rather than statistical noise around a random walk, consistent with Finding 1’s rejected permutation null. A simulation calibrated to these parameters, together with a mixture parameter $\pi_{\text{pure}} \approx 0.071$ accounting for pure-addition sessions (survivor ratio = 1), is best judged out of sample: a 70/30 train/test split gives held-out KS=0.18, $p=0.045$ (train $n=137$, test $n=60$). Unlike the smaller-sample fits, this is no longer a clean pass: across 8 train/test seeds the held-out p -value ranges [0.0003, 0.44], rejecting in 6/8 seeds. This is a downgrade: the expanded sample now typically shows a detectable discrepancy between held-out empirical and simulated survivor-ratio distributions (Figure 8), so the out-of-sample check should be read as marginal rather than confirmatory. The in-sample fit, always the more informative signal here, sharpens the same conclusion: the in-sample KS=0.21 ($p=<0.0001$) formally *rejects* exact distributional equality, so the model captures the coarse shape but is further from the true data-generating process once heterogeneity across 23 repositories and 20 developers is pooled into a single fit. That is exactly the kind of unmodeled repository/developer variation the multilevel model in Finding 2 quantifies directly ($vc_{\text{repo}}=0.132$, $vc_{\text{developer}}=0.062$).

Finding 5—Heavy-tailed session length. Session lengths are heavy-tailed: median 1 commit (50.9% of sessions are a single commit), mean 3.0, maximum 45. Figure 9 shows the empirical distribution with a log-normal fit, confirming that while most sessions are short, a substantial tail of long sessions drives mean complexity.

Finding 3, in full—pre-AI-adoption human baseline (descriptive, not a controlled comparison). The headline numbers are in §5.2; this is the construction and the full caveat list. The cutoff is derived from data, not assumed: scanning every repository in the same organization for the earliest commit anywhere carrying *any* AI-coding-tool co-author trailer (Claude, Copilot, Cursor, Codex, ChatGPT, and others) gives 2025-09-18; subtracting a 3-year safety margin sets the

cutoff at 2022-09-19. The 23 repositories used for the AI-tier findings turn out to have essentially no history before that date—they are recently-created repositories, a byproduct of the same qualification rule that selected them for the AI-tier analysis in the first place. That rule admits a repository on ≥ 15 Claude-co-authored commits counted across *all* branches, whereas extraction then reads only each repository’s *dominant* branch; a released per-repository commit count can therefore sit below the qualifying threshold without contradicting it—two denominators, both correct, and `scripts/discover_cohorts.py` derives each reproducibly. The control set consequently comes from a different, older, disjoint cohort of 122 repositories in the same organization with genuine pre-cutoff activity.

Developer identity is a curated roster, not a heuristic. Git author identities do not correspond to people: one person appears as a GitHub-noreply handle, a work address, a personal address, and sometimes a contractor account. Merging them with a fuzzy-matching heuristic and reporting the result as a contributor count does not work: the heuristic under-merges, inflating the headcount and splitting individuals across several labels. This analysis therefore uses a human-reviewed roster. `scripts/resolve_author_aliases.py --emit-candidates` writes a table of every raw identity together with the identity evidence behind its provisional cluster; a human marks each row keep-or-drop and assigns a person label; `--from-reviewed` converts that into a *strict* allowlist, so an identity absent from the roster is excluded from the cohort rather than merely left unmerged. The review covered 155 raw git identities and resolved them to the 62 people reported here, dropping one as unattributable. Merging aliases also fuses sessions that a heuristic would split across a single person’s two identities, which is why the control session count sits below what an uncurated extraction reports even though the commit count does not.

Curation was blind to outcomes by construction. The candidate table carries no survivor-ratio, churn, or ψ column and is assembled from identity evidence alone. This matters more than the headcount: a roster curated while looking at per-developer results would stop being a control, which is the one failure mode that would make this comparison circular rather than merely confounded. The choice has a plain cost: the roster maps real names and email addresses to person labels, so it cannot be released, and a third party therefore cannot rebuild this cohort from the artifacts we publish. What we can publish is the audit trail as counts (155 raw identities in, one dropped, 62 people out) and the salted `dev_`-prefixed labels in the released data, which are now stable across both cohorts and so support the paired analyses below.

To keep the two eras comparable, the control window (2021-11-08 to 2022-09-19) is sized to match the AI-tier data’s own 315-day observation span. Applying the extraction pipeline with `require_ai_trailer=False` (keeping all commits, still excluding any matching an AI-tool trailer as a defense-in-depth check even though none are expected before the cutoff) and grouping consecutive commits by the same developer into sessions yields 9,395 control sessions (17,694 commits). That is far more than the 401 AI-tier sessions, since we did not curate or subsample repositories, only bound the time window. The resulting mean non-pure survivor ratio is $\psi_{\text{human}} \approx 0.39$, and the churn-decay rate is $\rho_{\text{human}} = 0.86$ (95% CI [0.82, 0.91]), similar in shape to the AI-tier $\rho \approx 0.77$, so the geometric front-loading pattern itself (Finding 1) is not obviously AI-specific.

The survivor-ratio *level* is a different story: ψ_{human} is far below both ψ_{Opus} and ψ_{Sonnet} (gap = -0.160 vs. Opus at $p < 0.0002$ and -0.163 vs. Sonnet at $p < 0.0002$; both sit at the resolution floor $2/(9,999+1)$ of the permutation test, meaning no resample reached the observed gap, not that the p is estimated at that value), and a multilevel model with repository and developer as crossed random intercepts confirms the gap survives adjustment (adjusted coefficient $+0.145$, $p = 0.00082$ for Opus vs. human; $+0.154$, $p = 0.00026$ for Sonnet vs. human). Unlike Finding 2’s AI-tier comparison, the tier effect here is *larger* than the repository and developer variance components, not smaller.

One comparability objection would otherwise explain the gap without any appeal to era at all: the two cohorts have very different session-length profiles. 63.9% of control sessions are a single commit, against 50.9% of AI-tier sessions, and a one-commit session’s survivor ratio is a much coarser quantity than a ten-commit session’s. The objection does not bite. Restricted to multi-commit sessions on both sides, the mean non-pure survivor ratio is 0.435 for the control against 0.593 for the AI tier—a gap of the same sign and comparable size to the pooled one, so the level difference is not an artifact of the control being singleton-heavy.

We are explicit about what this cannot show: the human and AI-tier samples differ by era, not just by who wrote the code—codebase maturity, review norms, tooling, and team composition all differ across the roughly three years separating them, and any of these could produce a survivor-ratio gap with no AI involvement at all. This is a sanity check, not a controlled experiment, and it supports no causal claim about AI’s effect on the dynamics we measure; it does say that the tier-agnostic pattern of Finding 2 is not simply an artifact of any two groups of commits looking alike on this proxy—something in the AI-assisted sessions (whatever the cause) does look different from the pre-AI baseline, even as the AI tiers do not differ from each other.

Within-era AI vs. non-AI: the one contrast that removes both confounds (suggestive, not established). Finding 3 concedes two confounds it cannot address—the people differ, and the eras differ. Both can be removed at once by staying inside the AI era and splitting on the commit trailer instead: take the window 2025-09-18 to 2026-12-31, keep the same roster on both sides, and separate commits that carry a Claude co-author trailer from those that do not. The two sides then share developers, calendar window, codebase maturity, review norms, tooling, and seniority; they differ in whether the work was delegated to an agent. Pansuriya et al. [2026] draw the same human/agent contrast on pull-request acceptance and review effort, which is the closest existing comparison to this design, though on review outcomes rather than churn dynamics. This yields 487 AI-assisted sessions (1,493 commits) against 1,874 non-AI sessions (4,914 commits), with 22 developers present on both sides. Pooled, the mean non-pure survivor ratio is 0.540 for AI-assisted work against 0.419 for the rest—the same direction as Finding 3’s era contrast, with neither of its two confounds.

Pairing within developer, so that each person is compared only against themselves, gives Table 3. We report it as suggestive, not established. First, the *sign test is null at every floor*: counting how many developers move in which direction never reaches significance. Second, Wilcoxon does reach it at the two lowest floors, but only because it weights magnitudes as well as directions—at the ≥ 15 -commit floor the deltas run from -0.078 to $+0.332$, so the positive movements are several times larger than the negative ones. That asymmetry is an assumption the sign test deliberately declines to make, and quoting the significant p -value without it would misrepresent which assumption carries the result. Third, this is eight tests across four floors and two statistics, and the two significant values sit at the two lowest floors—the ones retaining the thinnest developers. Fourth, and least fixable by any amount of data: *task selection is uncontrolled*. This compares work developers chose to delegate against work they chose to keep, and there is every reason to expect those to differ in kind.

What the design does buy, and what the pre-AI pairing below does not, is that the direction no longer rests on thin data. It survives the ≥ 50 -commit floor (5 of 7 pairs, median $\Delta\psi = +0.127$), and the best-measured pair in the cohort—the developer with the most commits on *both* sides, 1,340 non-AI against 444 AI-assisted—moves with the majority at $\Delta\psi = +0.127$. Nothing here licenses a causal claim: the trailer is not randomly assigned, and the task-selection objection stands untouched.

Table 3: Part C: within-era AI vs. non-AI survivor ratio, paired within developer, at four minimum-commits-per-side floors. “AI higher” counts developers whose AI-assisted sessions show the higher mean non-pure survivor ratio. The sign test is null at every floor; Wilcoxon reaches significance only where the magnitude asymmetry is largest.

Commit floor	Pairs	AI higher	Sign p	Wilcoxon p	Median $\Delta\psi$
none	21	14	0.19	0.022	+0.127
≥ 15	14	10	0.18	0.011	+0.094
≥ 30	9	6	0.51	0.098	+0.061
≥ 50	7	5	0.45	0.11	+0.127

Pre-AI within-developer pairing: evidence *against* the between-group gap. Because both cohorts now share a salt and a roster, a person carries the same dev_ label in the pre-AI and AI-era datasets, which makes it possible to pair Finding 3’s era contrast within individuals and so remove its person confound—though not its era confound, since the two sides remain three years apart. 12 people have non-pure sessions on both sides. We report this result because it undercuts the between-group gap, not because it supports it.

Taken at face value the direction agrees with Finding 3: 9 of 12 developers show a higher survivor ratio in their AI-era sessions, median $\Delta\psi = +0.117$, though the sign test does not reach significance

($p=0.15$; Wilcoxon $p=0.034$, subject to the same magnitude-asymmetry caveat as above). That agreement is carried by developers with little data, and it does not survive requiring some. At a ≥ 50 -commit floor on both sides only 4 pairs remain, the majority disappears entirely (2 of 4, sign $p=1$), and the median delta collapses to $+0.017$ —an order of magnitude smaller than the pooled between-group gap of -0.160 . The single best-measured pair does move with the majority (322 pre-AI against 257 AI-era commits, $\Delta\psi=+0.320$); among the four best-measured pairs the direction splits evenly, so nothing here corroborates a level shift of the size Finding 3 reports.

A structural asymmetry compounds this and is the reason we report direction only, with no effect size or interval. The two datasets do not have equivalent inclusion rules: the control keeps *every* commit a person made in the pre-AI window, whereas the AI-tier dataset keeps only their Claude-co-authored ones. Each pair therefore compares different *slices* of one person’s work, which is legitimate for a per-session shape metric like the survivor ratio but makes any volume or productivity read across the two sides meaningless—the worst per-developer imbalance here is roughly 33-to-1, which reflects how little of that person’s output was delegated, not a collapse in what they produced. Read together with the within-era contrast above, the *within-person* evidence for a survivor-ratio difference is weak, and weakest exactly where it is best measured.

Finding 2’s four robustness checks in full. The main text counts the four checks that reinforce the capability-gap null; this is the detail behind them. (i) *Unmeasured confounding*. An E-value sensitivity analysis [VanderWeele and Ding, 2017] returns a point E-value of 1.10 and a confidence-limit E-value of 1.00: an unmeasured confounder would need associations of only that magnitude with both tier assignment and survivor ratio to account for the observed gap. The direction of this check needs care: a low E-value is unremarkable for a null, and we report it to bound how much a confound could be *hiding* an effect, not as evidence that one exists. (ii) *Task type*. Stratifying on the coarse task-type category and pooling within strata (4 strata) barely moves the estimate: adjusted gap $+0.002$ against a naive -0.003 ($p=0.96$), so the tier/task-type entanglement Limitations flags does not by itself manufacture the null.

(iii) *Decay rate, not only level*. The survivor ratio is a level; the churn-decay rate is a shape, and the tiers are indistinguishable on that too ($\rho_{\text{Opus}}=0.80$ vs. $\rho_{\text{Sonnet}}=0.75$, gap 0.043 , $p=0.54$, over the $n=123$ sessions long enough to fit a rate). (iv) *Repository and developer structure*. A multilevel model with crossed repository and developer random intercepts gives an adjusted tier coefficient of $+0.009$ (95% CI $[-0.070, +0.088]$, $p=0.82$), while the variance components ($\text{vc}_{\text{repo}}=0.132$, $\text{vc}_{\text{developer}}=0.062$) are an order of magnitude larger than any plausible tier effect. That last comparison is the substantive content of the null: in this data, which repository and which developer a session belongs to matters considerably more for its survivor ratio than which model tier wrote it. None of the four addresses model-version pooling, which is why the re-test below exists.

Finding 2 re-tested on an ordered capability scale (the null survives, but it is a *bounded* null). Finding 2’s tier variable is coarse in a way that could manufacture its own null. The label “Opus” collapses eight distinct commit-trailer strings and “Sonnet” four, so five Opus versions and three Sonnet versions are pooled into two bins—Opus 4.5 sits in the same bin as Opus 4.8. The within-bin capability spread is plausibly comparable to the between-bin contrast the test is trying to detect, and that would attenuate a real gap toward exactly the null we observe. This is also the first explanation a reader will reach for on seeing Part B’s large, clean capability effect (partial $\eta^2=0.497$ on a two-model comparison) beside Part C’s null, so we test it.

We therefore re-run the discriminator as a monotone trend over an ordered capability scale instead of a two-bin contrast. Each session is assigned its plurality (*family*, *version*) cell, giving 8 cells over 343 sessions. That total coincides with the version count just quoted and so invites a wrong reading: the cells are four Opus, three Sonnet, and one Haiku—not the five-plus-three the pooling above would suggest. Opus 5 appears in the cohort’s trailers but is the plurality cell of no session, so it contributes none; and Haiku, which the tier contrast never sees at all, contributes one. The cells are ranked two ways, each serving as the other’s sensitivity analysis: *family-major*, placing every Sonnet cell below every Opus cell and ordering by version within a family, and *version-major*, ordering strictly by version number across families. The null survives both. Spearman rank correlation between capability and survivor ratio is $r_s=+0.012$ ($p=0.82$) under the family ordering and $r_s=-0.008$ ($p=0.88$) under the version ordering—written r_s , not ρ , since ρ is the churn-decay factor throughout Part C; adding capability rank as a linear term to the same crossed repository/developer

multilevel specification used in Finding 2 gives per-step coefficients of $+0.003$ ($p=0.75$) and -0.011 ($p=0.38$). Version pooling is therefore *not* the explanation for Finding 2’s null: resolving the labels to versions and testing for a trend across them recovers no effect either.

That conclusion is a bounded null rather than a demonstrated zero, and needs the same caveat Part B’s underpowered nulls receive. The family-ordering adjusted interval is $[-0.016, +0.022]$ per rank step; accumulated over the 7 steps separating the weakest cell from the strongest, it still admits a total swing of up to 0.155, comparable in size to the human-vs-AI gap of -0.160 that Finding 3 does detect. The version ordering bounds it no more tightly ($[-0.036, +0.014]$ per step), so neither ordering converts the null into a narrow one; they agree on the sign being undetermined, not on the effect being small. What we can say is that no capability effect *large enough to dominate* the survivor ratio survives this test; what we cannot say is that the effect is zero. Read together with Part B’s large measured capability effect on the same kind of contrast, this is a fact about the proxy rather than about the models: the survivor ratio separates *whether an agent wrote the code* (ψ_{human} sits far below both tiers, and that gap survives crossed repository and developer random effects) but not *which* agent wrote it, being insensitive to precisely what a solve rate detects.

Two features of the per-cell breakdown explain why r_s sits so near zero. The per-cell means are *not monotone* in either ordering: the newest Sonnet cell ($\psi=0.446$) sits below an older Sonnet cell ($\psi=0.569$). So the trend statistic is averaging over non-monotone movement rather than detecting a weak monotone one. And the single Haiku cell sits at $\psi=0.352$ on only 8 sessions, essentially at the human baseline of 0.39: suggestive of a step at the very bottom of the capability range, on far too little data to claim one.

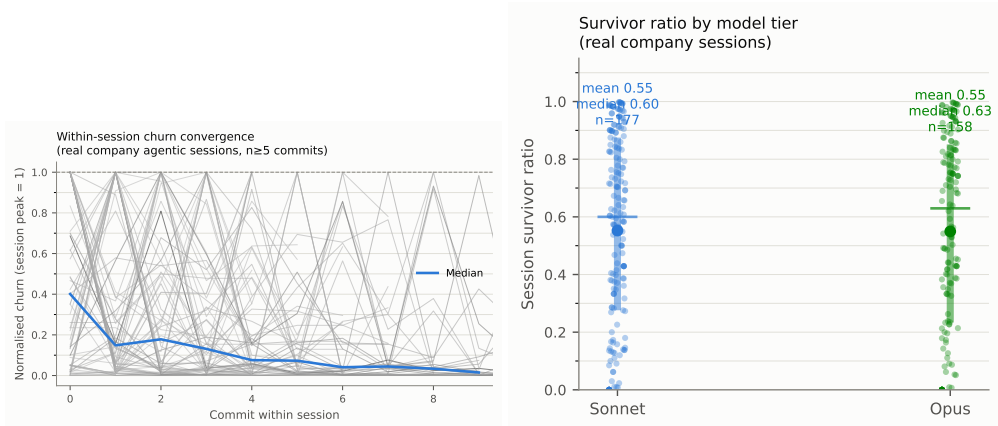


Figure 7: Part C production sessions, cited from Findings 1 and 2 (§5.2). **Left (Finding 1):** normalised within-session churn trajectories ($n \geq 5$ commits); churn concentrates in the first commit, decaying geometrically thereafter. **Right (Finding 2):** survivor ratio by model tier (dot = mean, bar = median)—the two tiers are statistically indistinguishable, a null that holds after adjusting for task type, repository, and developer.

G Extended Discussion

G.1 Related Work: Empirical Systems and Test-Time Compute

These are the empirical lines the body’s positioning refers to (§2). Multi-agent code generation (ChatDev [Qian et al., 2024], MetaGPT [Hong et al., 2023], AgentCoder [Huang et al., 2023]) and self-refinement work [Madaan et al., 2023, Welleck et al., 2023, Shinn et al., 2023] are empirical, without a structural criterion of this kind. Test-time compute scaling [Brown et al., 2024, Wang et al., 2022, Snell et al., 2024] shows empirically diminishing returns [Wu et al., 2024], which our refinement game predicts from first principles; raising the ceiling has required weight-level training [DeepSeek-AI, 2025]. A 17-hour, billion-token autonomous coding loop on a fixed harness [Khan, 2026] illustrates the same ceiling practically.

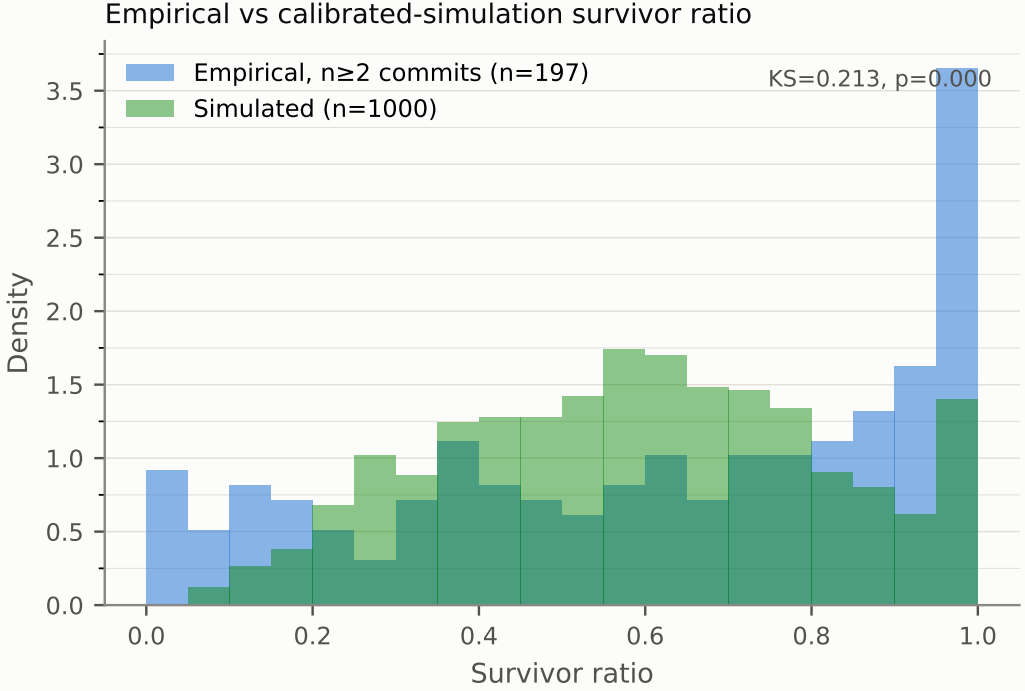


Figure 8: Empirical vs. simulated survivor-ratio distribution. Simulated sessions use the fitted geometric churn-decay (log-normal) model ($\rho=0.77$, $\sigma=2.39$) with a mixture component for pure-addition sessions ($\pi_{\text{pure}}=0.071$). The simulation captures the coarse shape but the annotated in-sample $\text{KS}=0.21$ ($p<0.0001$) formally rejects exact equality, and the held-out $\text{KS}=0.18$ ($p=0.045$, seed-sensitive across $[0.0003, 0.44]$, rejecting in 6/8 seeds) is now marginal rather than a clean pass—the gap likely reflects repository/developer heterogeneity pooled into a single fit (see Finding 4).

G.2 Could Decomposition Realize the Protocol Implicitly?

A natural objection to §3’s scope claim: a task decomposes into many sub-specifications (e.g., a ticket’s individual `FAIL_TO_PASS` tests), a refinement loop concentrates successive attempts on the still-failing ones, and one might read this as an implicit reweighting D_t with an implicit confidence vote α_t . We grant the spirit: Part B’s diagnostic feedback literally redirects the agent onto residual failures, a primitive attention-reweighting over sub-specs. But three features the transferred theorems actually require remain absent.

First, AdaBoost reweights a *fixed* population, whereas decomposition *generates* sub-specs on the fly; in the language of Prop. 3 this dynamic expansion is closer to the class growth of regime (ii) than to fixed-distribution reweighting. Second, “attend to the failing test” is a 0/1 shift, not the multiplicative-exponential update $w_i \leftarrow w_i \exp(-\alpha_t[\cdot])$ whose Z_t -factors telescope into the $e^{-2\gamma^2 T}$ bound; a generic failure-focus yields “the agent fixes tests at the rate its competence allows,” not the boosting rate. Third, and most structurally, AdaBoost *retains* all T learners and combines them by the weighted vote $f = \sum_t \alpha_t h_t$, whereas a refinement loop *composes* patches ($R_T \circ \dots \circ R_1$): each patch subsumes its predecessor rather than standing beside it to be voted on. There is therefore no additive margin for an implicit α_t to weight; the closest genuine analog, best-of- n patch selection, is a within-round argmax , not a cross-round vote.

We treat the mapping as *boosting-inspired*, not *boosting-exact*: deploying the exact protocol would restore the transferred theorems by construction, but at the cost of the fidelity we aim to preserve, since the real loops we set out to model do not run it. That tradeoff is inherent to a descriptive study of existing harnesses, not to the framework itself; §G.7 outlines what a harness that implements the Protocol outright, rather than approximates it, would look like as a design principle in its own right.

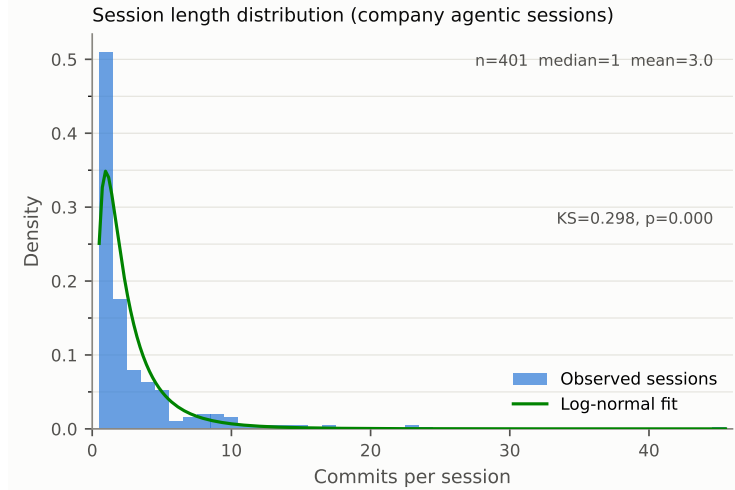


Figure 9: Session-length distribution (commits per session) with a log-normal reference overlay (descriptive; a KS test rejects exact fit). Median 1, mean 3.0, max 45. The heavy tail reflects long sessions that drive most of the structural change in the repositories.

G.3 Guidance for Coding in the Wild (full)

The main text gives the two headline recommendations; the full set of five follows, derived from the measured edge of Part B and the churn dynamics of Part C. We state these as working guidance consistent with our evidence and the inference-scaling literature, not as proven prescriptions: they rest on a $K=55$ single-vendor, two-tier ablation (Part B) and observational production data (Part C), and should be read as hypotheses a practitioner can cheaply test on their own workload rather than laws.

1. **Spend on the first attempt, not the refinement loop.** The single most actionable finding is that re-invoking an agent on its own output usually changes nothing: 80.7%/69.1% of trajectories never move after round 0, genuine no-ops run 52–59%/13–21% per round (Fig. 6), and per-round improvement decays to $0.097 \rightarrow 0.036 \rightarrow 0.033$ and $0.156 \rightarrow 0.068 \rightarrow 0.029$. Almost all of the value is realized on the first attempt, so better context, retrieval, and model choice for round 0 dominate a longer loop. The reassuring half of the same measurement: extra rounds are rarely *harmful* either (at most 2.9% of rounds lower quality), so a loop is wasteful rather than dangerous—and for a strong model the early rounds are the ones where an edge cannot be ruled out (Appendix E.1), which is where any budget for refinement should go.
2. **The verifier is the gradient—make it fast, executable, and specific (theoretical argument, not yet confirmed by our own ablation).** In boosting terms the verifier supplies the residual-error signal the loop descends on; in engineering terms your test suite, type checker, and runtime errors *are* the learning signal, the same lesson that process- and outcome-level verifiers carry in the reasoning literature [Lightman et al., 2023]. Part B’s own controlled ablation does *not* confirm this: diagnostic test-failure feedback did not significantly outperform blind re-sampling or independent redraws at $K=55$. Late solves occur at broadly similar rates under all three modes, and the feedback-mode effect is a small, underpowered null (Cohen’s $f=0.10$, 11% power). What our data *does* support is the weaker claim that verification quality is a lever worth caring about in principle (weak or absent verification structurally cannot supply a residual-error signal at all), not the stronger, specific claim that diagnostic feedback beats blind resampling in practice. We test only an agent critiquing its own output against a narrow test-failure signal here, not an architecturally separate reviewer model reading the same signal (Limitations); the two are not interchangeable, and our edge measurement should not be read as evidence against the latter.
3. **Reach for capability before orchestration.** The frozen-weight ceiling (Cor. 4) says a given model saturates at a quality $V^*(W)$ no loop can pass, and Part B shows the stronger model wins by clearing more tasks on the *first* try, not by refining better. Elaborate multi-agent refinement

pipelines built to compensate for an under-powered model will hit that ceiling; upgrading the model is often the cheaper path to higher quality on hard tasks.

4. **Distinguish exploring under the ceiling from raising it.** Two interventions are commonly both called “scaffolding” but behave differently. Re-sampling, “think again,” and more rounds *explore within* a fixed reachable class and saturate at $V^*(W)$; giving the agent genuinely new reach (additional tools, code search/retrieval, the ability to run tests, or task decomposition) can *expand* the reachable class $\mathcal{H}_t(C)$ and lift the ceiling (§G.4). When a loop has stalled, ask which kind of change you are making: only the second kind can move a hard plateau. This is also why Part B’s null feedback-mode effect does not cut against the theory: independent, blind, and diagnostic feedback are all within-class exploration variants (same weights, same tools, only the framing of feedback changes), so the theory’s own distinction does not anticipate a large gap between them.
5. **Steer early; stop on green.** Production churn is front-loaded (Part C, $\rho \approx 0.77$): the first commits make the structural decisions and later ones only polish, so an approach that is wrong at the outset is rarely repaired by subsequent edits—review and redirect early, when it is cheap. At the other end, stop as soon as the verifier passes: carrying the best passing artifact forward and halting on success avoids the failure mode where a loop instructed to “keep improving” regresses an already-correct solution. Our stopping rule (Algorithm 1) is a deterministic margin gate ($\epsilon_t \geq \frac{1}{2}$); a complementary line treats stopping as probabilistic inference—cost-sensitive sequential hypothesis testing over a belief about correctness [Papamarkou et al., 2026]—which is the natural refinement when verification is expensive and verifiers are informative but imperfect.

A final governance takeaway, expanded in §6 and Appendix G.4: the quantity to monitor is not whether model weights are frozen but whether the agent’s scaffold is self-expanding.

G.4 Implications for Recursive Self-Improvement (full)

The Stationarity Dichotomy (Prop. 3) states that iterative self-modification behaves in one of two ways. (i) It saturates into strict diminishing returns whenever the agent’s *effective hypothesis class*—the set of patches it can reach—is held fixed and quality is bounded. (ii) It escapes diminishing returns only if that class keeps expanding enough to sustain a positive edge. This converts the informal debate over an “intelligence explosion” into a concrete, checkable question: is the system’s reachable class stationary?

The whole-system view. Picture a craftsman whose hands are fixed but whose workshop is not: a sharper tool or a better-organised bench lets the same hands reach further, yet no rearrangement of the workshop makes the hands themselves more dexterous. It helps to view the self-improving system in the same way, as a pair $\mathcal{S} = (W, C)$, where W denotes the (frozen) parameters of the underlying model and C is the mutable code and scaffolding it operates (e.g., prompts, tools, retrieval, verifiers, and orchestration). Agentic coding is precisely the setting in which the system may rewrite C but not W .

It is tempting to infer that freezing W guarantees the safe, saturating regime (i). Under the model we now make explicit, this is *false*. We posit that the effective hypothesis class $\mathcal{H}_t(C)$ from which patches are drawn depends on C , and that rewriting the scaffold can expand $\mathcal{H}_t(C)$ without touching a single weight (assumption (M1) of Cor. 4). A better verifier, tool, or decomposition can unlock patches the previous configuration could never propose. Concretely: an agent stuck on a flaky integration test might write and run a small isolation script mid-session, a debugging tool it did not have at round 0. Once written, that script is available to every later round, so patches the agent could not previously validate (because failures were masked by flakiness) become checkable: $\mathcal{H}_t(C)$ grows to include edits unreachable at $t=0$, with W untouched throughout. This is the modeling premise behind recent calls to treat the harness as a first-class, separately optimizable layer [He et al., 2026]. That configuration is not hypothetical. Inspect, the open evaluation framework published by the UK AI Security Institute [AI Security Institute, UK, 2024], ships a `react()` agent that runs a reason-act-observe loop with `bash()` and `python()` tools inside a Docker sandbox, and can host external coding agents (Claude Code, Codex CLI, Gemini CLI) directly—so the agent that writes and runs its own diagnostic mid-run is the default arrangement of a harness built for frontier evaluation, not a thought experiment. Two consequences pull opposite ways. Constructively, the scaffold there is a *declarative, versioned* artifact—a solver chain, a tool list, a Dockerfile—which is the precondition

for auditing a reachable set at all: stationarity of $\mathcal{H}_t(C)$ is not checkable unless the scaffold is first enumerable. Against that, the declared unit of measurement is a *task* (dataset, solver, scorer), and nothing in that unit records whether the reachable set expanded while the task ran. The scaffold is thus versioned as experimental setup rather than tracked as the property whose expansion (M1) identifies, the same gap the published frameworks leave at the policy layer (below), one level down in the instrument. Such self-modification violates the stationarity assumption behind Prop. 3(i), so a frozen-weight system can still exhibit the non-monotone, jumpy trajectory of regime (ii).

Frozen weights plausibly impose an ultimate ceiling. We assume there is a best quality $V^*(W)$ achievable by *any* scaffold the model can construct within its reachable program space (assumption (M2) of Cor. 4). Once scaffold rewrites can no longer expand $\mathcal{H}_t(C)$, the system falls back into regime (i) and saturates at a value no greater than $V^*(W)$. Genuinely unbounded RSI therefore requires raising $V^*(W)$ itself—modifying the weights or architecture—together with co-evolving the evaluation. Hernandez and Brown [2020] document how much of that raising has historically come from algorithmic progress rather than scale alone, which is why $V^*(W)$ is best read as a property of a particular checkpoint rather than a constant of the field. The practical upshot for anyone exploring a self-improving system is a diagnostic: determine whether the reachable class is effectively fixed (predictable, saturating) or expanding (potentially runaway), and do *not* treat frozen weights as sufficient evidence of the former. Neither (M1) nor (M2) has been empirically measured; the conclusion is conditional on both holding.

The generalization bound (Thm. 12) sharpens the same point from the evaluation side: an AI optimizing against a static set of N specifications overfits, so pushing back the $O(\sqrt{d/N})$ ceiling requires dynamically expanding the test suite. Escaping saturation thus demands expansion on *both* sides—the hypothesis class and the tests.

Concretely, the framework identifies the mechanisms that move a system from regime (i) into regime (ii):

- **Self-Evolving Tests and Cost Functions:** Instead of a fixed dataset of specifications, the AI could generate increasingly complex tests (an automatic curriculum, as in Wang et al., 2023) or evolve its own structured loss functions. This continuously shifts the generalization target and prevents plateauing.
- **Goal-Setting Exploration:** Rather than solely minimizing residual error on existing tasks, agents could autonomously set new, orthogonal objectives. This would force structural shifts in the code-base, potentially opening up new capability dimensions beyond simple patch refinement.
- **Reward Hacking and Awareness:** Explosive trajectories might involve the AI recognizing and exploiting proxies in its evaluation metrics (reward hacking, a recognized failure mode of proxy objectives Amodei et al., 2016, Skalse et al., 2022). A robust recursive protocol must be aware of its own margin distribution and actively penalize the artificial optimization of the structured distance $L(Y, \hat{Y})$ when it degrades true functional quality.

We read each mechanism as, in the language of Prop. 3, a way of expanding the effective hypothesis class $\mathcal{H}_t(C)$ or the evaluation target. Absent them, the stationarity assumption holds and Prop. 3(i) forces the system to plateau.

Relation to test-time compute, and why test-time training is a different case. The bounded regime our theory identifies matches the empirical inference-scaling literature: repeated sampling and compute-optimal test-time allocation raise quality with diminishing returns against a fixed model [Brown et al., 2024, Snell et al., 2024, Wu et al., 2024]. In our terms, scaffold- and sampling-level effort explores within a fixed reachable class and saturates at $V^*(W)$; moving the ceiling has required weight-level training such as reinforcement learning for reasoning [DeepSeek-AI, 2025]. We read this as consistency, not confirmation—the inference-scaling studies were not designed to test our theorems.

The criterion does, however, force a distinction that the test-time-scaling literature does not consistently draw (§2), and the safety reading depends on it. *Test-time training* updates parameters at inference: from a self-supervised task on the test input [Sun et al., 2020], from retrieved neighbours [Hardt and Sun, 2024, Hübötter et al., 2025], from the in-context examples of the target task [Akyürek et al., 2025], or by making the hidden state itself a learned model [Sun et al., 2025]. Under Cor. 4 this is not more search beneath $V^*(W)$; it changes W , so it changes the ceiling, and the

frozen-weight premise simply does not hold for it. That places it in regime (ii) by construction rather than as an empirical question.

First, the empirical fact that test-time training buys large gains where prompting plateaus [Akyürek et al., 2025] is expected under the criterion rather than surprising: the two interventions are not competing points on one compute axis but moves in different regimes. Second, an evaluation protocol that reports only “total test-time compute” pools the two and can therefore attribute a ceiling-raising effect to bounded search. Third, and most consequential for governance, “are the weights frozen?” is the wrong single question: it catches test-time training, which visibly writes to W , while missing scaffold self-expansion, which does not touch W at all and is the mechanism (M1) identifies. An audit needs both.

What the published frontier-safety frameworks actually check. The claim that weight-immutability is the wrong invariant is not aimed at a strawman, but the position it corrects is more careful than “check whether the weights changed,” and stating it precisely matters. Current frontier-safety frameworks already elicit capability *with* scaffolding: Google DeepMind applies “appropriate scaffolding, inference compute, and other augmentations” so as to “assess the capabilities of systems that will likely be produced with the model” [Google DeepMind, 2026], and OpenAI evaluates “with the best presently-available scaffolds” [OpenAI, 2025]. What is indexed to the checkpoint is not the elicitation but the *re-assessment trigger*: assessment is occasioned by training-side events—the completion of a post-training run [Google DeepMind, 2026], or $4\times$ “Effective Compute” or “six months of accumulated post-training enhancements” [Anthropic, 2026]—so scaffold-driven capability gain enters as a fixed margin around a checkpoint rather than as a tracked property of the deployed loop. Both developers name the residual as an open problem in their own terms. OpenAI records that “given the continuous progress in model scaffolding and elicitation techniques, we regard any one-time capability elicitation in a frontier model as a lower bound, rather than a ceiling, on capabilities that may emerge in real world use and misuse” [OpenAI, 2025]; Anthropic, having replaced its prescriptive elicitation buffer with an outcome-based requirement, states that “the science of evaluations is not currently mature enough to make confident predictions about the precise buffer we should require between current models and a Capability Threshold” [Anthropic, 2026]. Cor. 4 says why no fixed buffer settles the question in general: under (M1) a scaffold rewrite moves $\mathcal{H}_t(C)$ *between* assessments, with no training-side event to occasion one, so the buffer would have to bound an expansion its own trigger cannot see. The invariant that would close that gap is stationarity of the reachable set, not immutability of W . One framework already reaches part of the way—OpenAI’s covers “any agentic system (including significant agents deployed only internally) that represents a substantial increase in the capability frontier” [OpenAI, 2025], which is a scaffold-side trigger, though a discretionary and frontier-indexed one rather than a check on whether a particular loop’s reachable set is expanding.

The nesting convention runs the other way in some of the literature: test-time training presented as one strategy within test-time compute [Muñoz and Yuan, 2025], and the main survey treating its tuning branch as offline rather than per-instance [Zhang et al., 2025b]. The taxonomy here is ours, asserted because the dichotomy requires it, not inherited from a survey.

Where the scaffold writes to the weights, and what bounds that instead. One arrangement closes the loop in the other direction, and it is the case assumption (M2) has to survive. SEAL trains a model to emit its own finetuning data, so a scaffold-level generation drives a persistent weight update [Zweiger et al., 2025]. That is not search beneath a fixed ceiling and it is not test-time training on the test input either: it raises $V^*(W)$ *using the scaffold as the instrument*. This is precisely why (M2)’s quantifier over “any scaffold the model can construct” is load-bearing rather than definitional—a scaffold licensed to write to W escapes the bound by construction, and Cor. 4’s frozen-weight premise simply does not hold for such a system. The route also carries a bound our dichotomy does not supply. SEAL reports performance on earlier tasks declining as self-edits accumulate—the catastrophic-forgetting failure mode of repeated weight updates—so sustaining the uniform edge Prop. 3(ii) demands is not merely a matter of being permitted to change W . Read alongside Cor. 4(ii), that leaves two distinct ceilings with different causes: ours from a fixed reachable class, theirs from interference between successive updates. An evaluator should expect to meet whichever binds first, and neither is visible to a checkpoint-level audit.

Descriptive regime vs. mechanistic cause. The empirical tracks support two claims to different degrees. The *descriptive* claim, that agentic coding occupies the bounded, saturating regime (i) of Prop. 3, is well-supported: churn decays and quality saturates (Findings 1, 3) exactly as the bounded regime requires. That this signature is *shared* with ordinary human editing is not a weakness of the descriptive claim; it is the observable content of “the system is in regime (i),” and is itself the safety-relevant finding. Current frozen-weight agentic coding refines like a bounded editor, not a runaway process. What the shared signature does *not* establish is the *mechanistic* claim, that a fixed reachable hypothesis class is the *cause* of the saturation, as opposed to generic editing dynamics. Attributing the regime to the mechanism requires the theory’s non-generic content: not merely that a more capable frozen model reaches a higher $V^*(W)$ (Cor. 4), but that $V^*(W)$ is *impassable by scaffolding alone*, moved only by weight-level change. The decisive experiment combines Part B’s harness with Part C’s scale, holding weights fixed while varying the scaffold rather than comparing tiers, which the present data motivates but does not deliver.

G.5 Bounded vs. Unbounded Metrics

An exam scored out of a hundred admits only so much improvement. However much revision remains, the marks still available can never exceed the marks not yet earned. So each further hour of study returns less than the last. A bounded metric enforces the same discipline, and it is the load-bearing assumption in our formalization of diminishing returns: it is what makes Prop. 3(i) a theorem rather than a conjecture. In the Refinement Game (Thm. 2), the quality function $V : \mathcal{Y} \rightarrow [0, 1]$ is bounded above by 1. Because of this ceiling the series $\sum_{t=1}^{\infty} \eta_t$ of positive improvements converges, so its *terms* η_t must shrink toward zero, mathematically enforcing strict diminishing returns. This is exactly case (i) of the Stationarity Dichotomy (Prop. 3) with ceiling $B = 1$.

Conflating two claims here would overstate what the formalism delivers. The *proved* claim needs boundedness and has no content without it. A *weaker* claim survives its removal, but only as an argument, not as a consequence of Thm. 2: if the system were evaluated on an unbounded, real-valued metric (maximizing transactions per second, minimizing latency toward an asymptotic zero, an open-ended reward score), the strict boundary breaks down, and the following three factors would still be expected to force plateauing—none of them formalized here, and the third not even a limit on capability but a corruption of the measurement:

1. **Overfitting Ceiling:** As shown in Thm. 12, generalization is bounded by the VC-dimension and the test suite size. Endless optimization of an unbounded metric against a static set of N specifications will eventually result in overfitting. Breaking this ceiling requires continuously generating increasingly complex tests.
2. **Gradient Depletion:** In functional gradient descent, optimizing further down the loss curve makes residual errors vastly more complex. A weak coding agent will eventually struggle to find meaningful orthogonal improvements, meaning its edge γ drops to 0 or becomes negative.
3. **Reward Hacking:** As noted above, unbounded metrics are highly susceptible to proxy exploitation [Amodei et al., 2016, Skalse et al., 2022]. The AI may artificially optimize the unbounded score while degrading true functional quality.

G.6 Remaining Limitations

The main text states the four most consequential limitations (the shared human-baseline decay, the edge measurement and its confounds, unchecked restarts, and the tier-assignment confound). The remaining items:

1. **Self-refinement vs. a distinct reviewer role (untested architecture).** Part B’s feedback-mode ablation (independent/blind/diagnostic) varies what a *single* agent is told about its own prior patch—it never introduces an architecturally separate reviewer or critic model that inspects the patch and returns its own feedback. Even the diagnostic arm’s critique is deliberately narrow (which FAIL_TO_PASS tests still fail, with a one-line pytest hint), chosen to avoid leaking the gold pass/fail bit to the generating agent, not to be the richest signal a reviewer could give. A generator-plus-reviewer pipeline is a materially different, plausibly higher-edge and less-correlated feedback channel that this work does not test.

2. **Binary quality labels.** Our theory is stated for a strict binary success criterion ($\{-1, +1\}$). The Lean development is in fact more general: the core identities are proved for arbitrary real-valued labels and hypotheses, and the binary structure is introduced only as an explicit per-theorem hypothesis (`h_binary`) precisely where the AdaBoost weight update requires it. While general code quality is multi-dimensional, the binary criterion aligns well with software engineering, where “multi-dimensional quality” effectively amounts to a conjunction of binary gates.
3. **Lean formalization scope and motivation.** The repository contains no axiom and no sorry: statistical components are encoded as explicit *hypotheses* rather than derived. Specifically, Theorem 12 takes the Schapire et al. [1998] margin bound as a hypothesis, sidestepping a complete VC-dimension and Rademacher-complexity library, and the deterministic refinement game (Theorem 2) supplies linearity and monotonicity of expectation as hypotheses. By contrast, the high-probability refinement corollary (Cor. 15) is proved against Mathlib’s `IsProbabilityMeasure` and finite union bound, and the stationarity dichotomy with its frozen-weight ceiling corollary is fully machine-checked.
4. **Simulation fidelity and benchmark scope.** Part A’s synthetic labels bound what it can show, and Appendix D states that scope rather than repeating it here. Part B is a regime characterization ($K=55$ tasks $\times$ 5 seeds, a 2×3 model-tier $\times$ feedback-mode ablation, single vendor, digest-pinned Docker harness), not a shortcoming: it probes the headroom $\times$ capability condition and finds a large, significant capability effect alongside an underpowered feedback-mode null, but it is not the primary empirical evidence. The task pool is additionally pytest-native by construction, so the two largest Lite repositories are absent for a reason internal to our tooling rather than to the benchmark (Appendix E gives the full disclosure). Importing SWE-bench’s per-repository test commands and restoring both repositories is the first thing a replication should do; it is also the one change that would move every Part B number reported here, which is why it is left to a replication rather than folded into this revision. A ready instrument exists: the companion evaluation collection to Inspect [AI Security Institute, UK, 2024] implements SWE-bench with grading delegated to upstream `swebench.harness.grading.get_eval_report` rather than re-implemented per-repository log parsing, which is exactly the import this item calls for.
5. **Unmeasured edge in production; survivor-ratio proxy.** Corollary 11’s exponential convergence requires each agent to have edge $\gamma_t > 0$ under adversarial reweighting onto hard specs. In Part C this edge is not measured on real agents; the guarantee is therefore a conditional prediction, not an empirically confirmed bound. Separately, the survivor-ratio proxy `|net lines|/total churn` conflates distinct session types: a session that *adds* 100 net lines and one that *deletes* 100 lines of dead code both receive the same ratio, though they represent qualitatively different outcomes—the proxy measures structural convergence, not functional correctness. Relative churn does have an established software-engineering pedigree—Nagappan and Ball [2005] show relative churn measures predict defect density—but it is validated there as a predictor of *defects*, not of the convergence we read into it, so the borrowing is partial.
6. **Stationarity of the refinement game.** Theorem 2 assumes that each agent improves quality by at least η_t in expectation. In practice, agents may occasionally degrade quality. Corollary 15 relaxes the strict monotone requirement to a high-probability bound. More fundamentally, stationarity is not merely a technical convenience: as formalized in the Stationarity Dichotomy, the question of whether the effective hypothesis class stays fixed is the precise dividing line between the bounded regime our theorems cover and the runaway regime they do not.
7. **The reasoning/tool-creation boundary is not operationally sharp.** Assumption (M1) treats scaffold expansion as a binary: either $\mathcal{H}_t(C)$ grows or it does not. In practice the line between reasoning *within* the current class and an action that *expands* it is blurry: writing a new unit test is ordinary reasoning about the existing code, but running it behaves like acquiring a new tool—patches that were previously unreachable because unvalidatable become checkable, without any change to W . The same ambiguity recurs for a debug script, a refactor that exposes a previously-untestable seam, or a cached search index. We do not have a principled test for where a given action falls on this continuum; (M1) is stated as a binary precisely because an operational boundary is missing, not because one is known and simply omitted.

G.7 Future Directions

- **Multi-class and structured prediction.** Extending to multi-label code quality (security, performance, readability) using SAMME [Zhu et al., 2009] or structured boosting.
- **Online and streaming protocols.** Adapting the framework to settings where artifacts arrive sequentially, using online boosting [Beygelzimer et al., 2015]. “Online” means streaming instances here, and is distinct from the two other senses this paper touches: composing an improvement process at inference time (Xue and Yang, 2026, §2) and continual weight adaptation across a stream of tasks (Zweiger et al., 2025, above). Only the last of the three moves $V^*(W)$.
- **Scale and diversity of empirical validation.** Extending Part C to multiple companies, larger session datasets, and harness-verified quality signals (true pass/fail via test suites) would provide stronger evidence and allow quantitative hypothesis testing against the theory’s predictions.
- **Optimal agent selection.** Developing efficient algorithms for selecting the next agent (weak learner) in the coding domain, where evaluating an agent’s error is expensive.
- **Generator-reviewer ablation.** A natural extension adds a fourth arm in which an architecturally separate reviewer model, given the same non-gold test-failure signal already available to the diagnostic arm, returns free-form critique—testing whether a distinct reviewer role sustains a positive edge where self-critique plateaus after round 0.
- **The multi-round weighted-vote run.** Appendix C settles the per-round edge for a reused pool in both directions (Prop. 7) but stops short of exhibiting a *run*: a schedule and confidence sequence written out over T rounds, which additionally requires excluding a perfect pool member since Cor. 11 assumes $\epsilon_t > 0$. Formalizing AdaBoost’s minimax duality in the converse direction—weak learnability $\Rightarrow$ interpolation with margin—would close the remaining half of the same picture.
- **Implementing the Protocol itself as a harness design principle.** Every empirical part in this paper studies harnesses that approximate the Agentic Boosting Protocol at best; none run its reweighting machinery (D_t, α_t) directly (§G.2). A harness that maintains an explicit distribution over a *fixed* spec or test-case pool, up-weights the specs the current draft still fails, and combines successive patches by a weighted vote rather than sequential composition would instantiate Algorithm 1 exactly rather than by analogy—restoring the transferred AdaBoost guarantees (Thms. 10, 12) by construction, and giving the boosting lens a literal, checkable referent past round 0, rather than only an inspired one.

H Formalization Index

Every result above is machine-checked in Lean 4 (Mathlib, no axiom and no sorry); these two tables map each one to the declarations that carry it, so a reader can go from a statement to its proof term. All declarations live in `lean_proofs/LeanProofs/`. Where a result’s content is statistical rather than algebraic it is encoded as an explicit *hypothesis* rather than derived, and the tables say so.

Table 4: Core results and the Lean declarations that carry them (principal name first, then supporting lemmas and non-vacuity witnesses). All declarations live in `lean_proofs/LeanProofs/`.

Result	Lean declarations
Thm. 10, training-error bound	<code>thm1_zero_one_loss_bound</code> (the bound as stated); <code>thm1_training_error_bound</code> for the underlying exponential-loss identity $\exp \text{ loss} = \prod_t Z_t$, with <code>z_decomposition</code> , <code>indicator_le_exp</code> , <code>exp_loss_pos</code>
Thm. 13, forward stagewise equivalence	<code>thm4_fsam_optimal_alpha</code> ; <code>thm4_fsam_global_minimum</code> , <code>exp_half_log</code>
Cor. 11, exponential decay	<code>cor2_exponential_decay</code> ; <code>z_le_sqrt_edge</code> , <code>sqrt_edge_le_exp</code> , <code>z_bound_from_edge</code>
Thm. 2, monotone-improvement refinement game	<code>thm5_refinement_game</code> ; <code>thm5_saturation</code>
Cor. 15, high-probability refinement	<code>corollary7_high_prob_refinement</code> ; <code>prob_inter_bound</code> , <code>sum_telescope</code> . Verified against Mathlib’s measure-theoretic library (<code>IsProbabilityMeasure</code> and the finite union bound)
Prop. 3, stationarity dichotomy	<code>prop8_saturation_forces_decay</code> (part i), <code>prop8_sustained_edge_diverges</code> (part ii)
Cor. 4, frozen-weight ceiling	<code>cor_frozen_weight_ceiling</code> , an instantiation of <code>prop8_saturation_forces_decay</code> at $B := V^*(W)$
Prop. 7, reused-pool dichotomy	Stated in the main text as the two halves formalized separately below: <code>thm_pool_reuse_horizon</code> (case i, $\epsilon_0 > 0$) and <code>thm_interpolating_pool_sustains_edge</code> (case ii, $\epsilon_0=0$), with <code>cor_pool_horizon_floor_excludes_interpolation</code> for their mutual exclusion. Exhaustiveness is immediate, since $\epsilon_0 > 0$ or $\epsilon_0=0$; see Props. 20 and 21 for the exact hypotheses
<i>Assumed as an explicit hypothesis, not derived:</i>	
Thm. 12, generalization bound	<code>thm3_generalization_bound</code> takes <code>MarginGeneralizationBound</code> as a hypothesis. Its statistical content—the Schapire et al. [1998] margin bound, VC dimension, and Rademacher complexity—is assumed, sidestepping a complete VC/Rademacher library

Table 5: Orchestration and diversity results of §C and their Lean declarations. Every row is machine-checked; the one item that is *not* formalized is recorded in the note below the table.

Result	Lean declarations
Prop. 5, best-of- k ceiling equality	<code>prop15_tight_ceiling_orchestration</code> (tight equality); <code>prop15_orchestration_ceiling</code> , <code>prop15_orchestration_dominates</code> for the weaker upper-bound-hypothesis version
Prop. 17, weighted aggregation beats the weak-learning baseline	<code>cor_soft_aggregation_beats_weak_baseline</code> ; built on the pure-analysis lemma <code>exists_ensemble_size_beating_weak_threshold</code>
Prop. 18, multi-round best-of- k	<code>thm_width_refinement_game</code> ; <code>cor_width_dominates_best_worker</code> , <code>cor_width_saturation</code> , with <code>exists_width_orchestrator</code> for non-vacuity
Prop. 6, bounded overlap characterizes a correct vote	<code>thm_overlap_iff_zero_error</code> (characterization); <code>cor_tight_overlap_iff</code> (m^* form), <code>thm_bounded_overlap_zero_error</code> (sufficient direction), <code>thm_overlap_boundary_sharp</code> with <code>exists_boundary_overlap_family</code> (sharpness at the boundary), <code>cor_overlap_markov_bound</code> (counting-only baseline)
Reweighted edge hypothesis via the density ratio	<code>lem_bounded_reweighting_edge</code> takes the concentration cap κ as given; <code>D_le_of_margin_bound</code> and <code>cor_rho_from_confidences</code> derive it from logged confidences, and <code>cor_reweighting_edge_from_confidences</code> states the κ -free conclusion. The sources spell κ as ρ and M as B , the names they carried before this paper separated those glyphs
Prop. 19, immediate reuse zeroes the edge	<code>thm_self_reuse_zero_edge</code> ; rests on the derived reweighting recursion <code>D_succ_eq</code> and <code>reused_worker_err_eq</code> , with <code>exists_zero_edge_reuse_instance</code> for non-vacuity
Prop. 20, reused-pool horizon	<code>thm_pool_reuse_horizon</code> , <code>cor_no_perpetual_pool_edge</code> ; rests on the span-collapse identity <code>lem_pool_span_collapse</code> , its loss form <code>zero_one_loss_eq_spanLoss</code> , and <code>alpha_opt_nonneg</code> , with <code>exists_positive_span_floor</code> and <code>exists_pool_reuse_horizon_instance</code> for non-vacuity and <code>exists_edge_worker_vs_any_round</code> for the absence of a per-round obstruction
Prop. 21, interpolating pools sustain the edge	<code>thm_interpolating_pool_sustains_edge</code> , with <code>cor_interpolating_pool_edge_every_round</code> for AdaBoost’s own reweighting; rests on <code>lem_margin_floor_of_spanLoss_zero</code> , <code>lem_span_coeff_sum_pos</code> , <code>lem_weighted_margin_eq</code> , <code>lem_D_nonneg</code> , with <code>cor_pool_horizon_floor_excludes_interpolation</code> for mutual exclusion with Prop. 20 and <code>exists_interpolating_pool_instance</code> for non-vacuity
Prop. 22, the two conditions are independent	<code>thm_edge_overlap_independent</code> ; from the witnesses <code>thm_edge_not_imply_overlap</code> and <code>thm_overlap_not_imply_edge</code> and the exponential-free reweighting steps <code>D_succ_fail</code> , <code>D_succ_pass</code>
Prop. 23, Condorcet bound for independent errors	<code>thm_condorcet_majority_bound</code> , over the pattern weight <code>wt</code> and vote error <code>voteErr</code> ; rests on <code>lem_wt_sum_one</code> and <code>lem_wt_tilt_sum</code> , with <code>exists_condorcet_instance</code> for non-vacuity
Prop. 24, the same bound with independence <i>derived</i>	<code>thm_condorcet_majority_bound_prob</code> , over the product measure <code>failMeasure</code> ; <code>lem_coord_iIndepFun</code> obtains independence from Mathlib’s <code>iIndepFun_pi</code> , <code>lem_pattern_measure</code> the pattern weight, <code>lem_failure_event_eq_voteErr</code> the event identification, and <code>lem_filter_decide</code> the pattern/failure-set bijection, with <code>exists_condorcet_prob_instance</code> for non-vacuity

Not formalized (the single residual item): the multi-round weighted-vote *run*—a schedule and confidence sequence exhibited outright, as distinct from the per-round edge that feeds it, which additionally requires excluding a perfect pool member since Cor. 11 assumes $\epsilon_t > 0$ (§C). The converse direction of AdaBoost’s minimax duality (weak learnability $\Rightarrow$ interpolation with margin) is likewise not formalized.